



LLAMA3 Hands-on

법률 고지사항

- Intel® 디지털 준비 프로그램 및 Intel® AI for Future Workforce 프로그램은 Intel Corporation에서 개발했습니다.
- © Intel Corporation. Intel, Intel 로고 및 기타 Intel 마크는 Intel Corporation 또는 자회사의 상표입니다. 다른 이름 및 브랜드는 다른 사람의 재산으로 주장될 수 있습니다. 프로그램 날짜와 수업 계획은 변경될 수 있습니다.
- Intel 기술에는 활성화된 하드웨어, 소프트웨어 또는 서비스 활성화가 필요할 수 있습니다.
- 모든 제품과 구성 요소는 안전을 보장 할 수 없습니다.
- 결과물은 추정되거나 시뮬레이션 되었습니다.
- Intel은 타사 데이터를 제어하거나 감사하지 않습니다. 정확성을 평가하려면 다른 출처를 참조해야 합니다.
- 당신이 투자한 비용과 그에 대한 결과물은 다를 수 있습니다.

LLAMA model 구성

Meta-Llama 확인

- <https://huggingface.co/meta-llama> 에서 Llama 모델 확인
- Llama 3.2: The Llama 3.2 collection of multilingual large language models (LLMs) is a collection of pretrained and instruction-tuned generative models in 1B and 3B sizes (text in/text out).
- Llama 3.2 Vision: The Llama 3.2-Vision collection of multimodal large language models (LLMs) is a collection of pretrained and instruction-tuned image reasoning generative models in 11B and 90B sizes (text + images in / text out)
- Llama 3.1: a collection of pretrained and fine-tuned text models with sizes ranging from 8 billion to 405 billion parameters pre-trained on ~15 trillion tokens.

Meta-Llama 3.2-3B-Instruct 확인

huggingface.co/meta-llama/Llama-3.2-3B-Instruct

 **Hugging Face**

meta-llama / **Llama-3.2-3B-Instruct**   like 887  Follow

 Text Generation  Transformers  Safetensors  PyTorch  8 languages

meta llama-3 conversational  text-generation-inference  Inference Endpoint

 arxiv:2405.16406  License: llama3.2

  Train  Deploy  Use this model

 Model card  Files  Community **52**

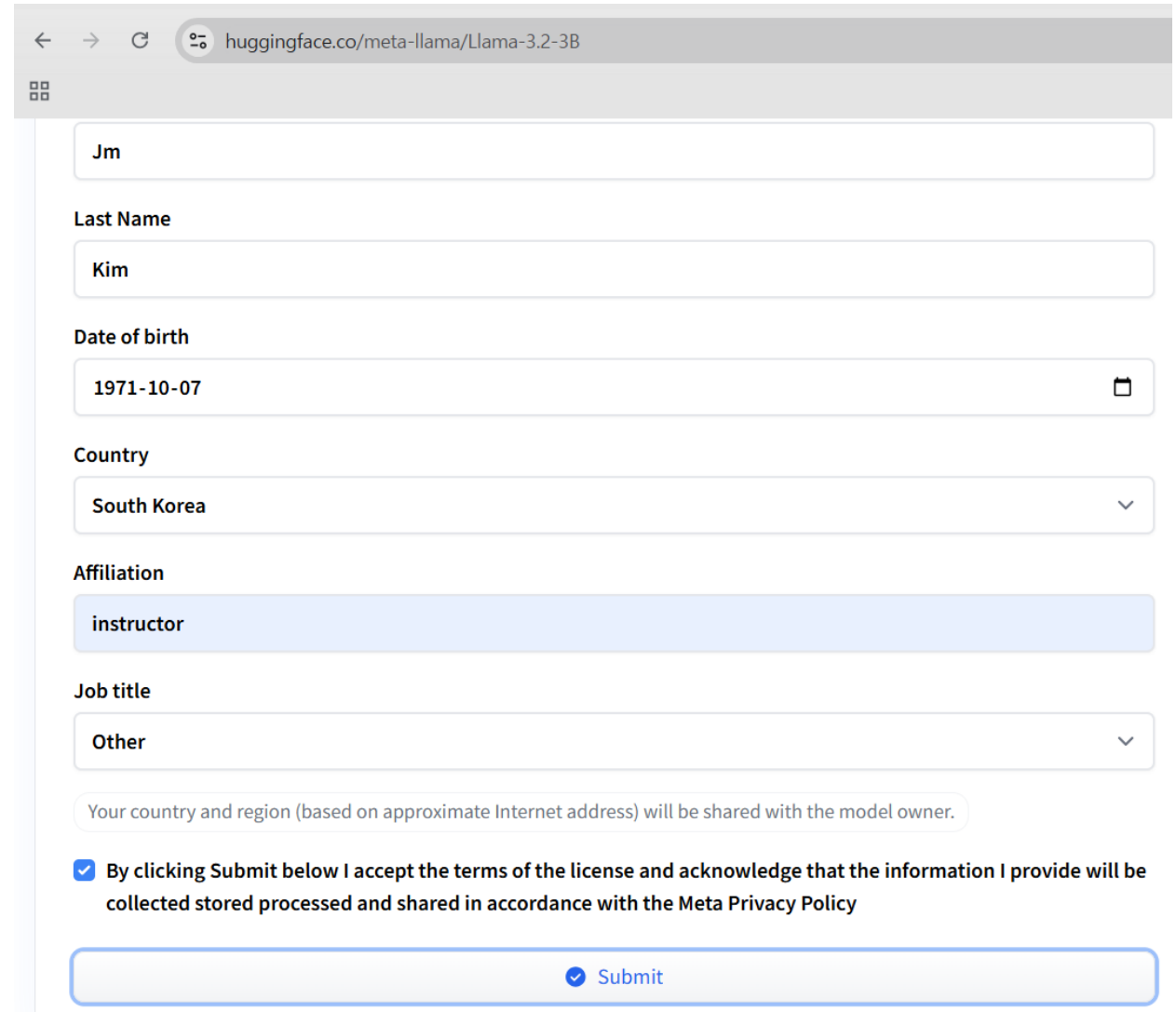
 Edit model card

 **Gated model** You have been granted access to this model

Downloads last month
1,169,342

Meta-Llama 3.2 라이선스 확인

사용하기 위해서 라이선스 확인이 필요



The screenshot shows a web browser window with the URL `huggingface.co/meta-llama/Llama-3.2-3B`. The form contains the following fields and options:

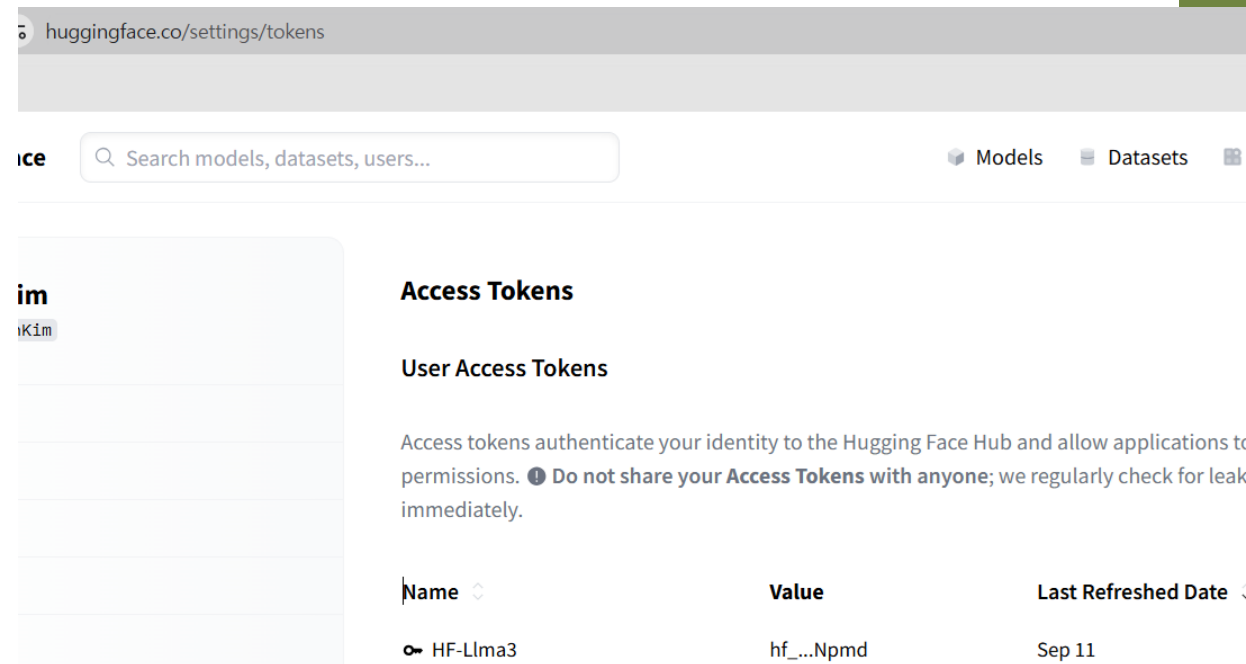
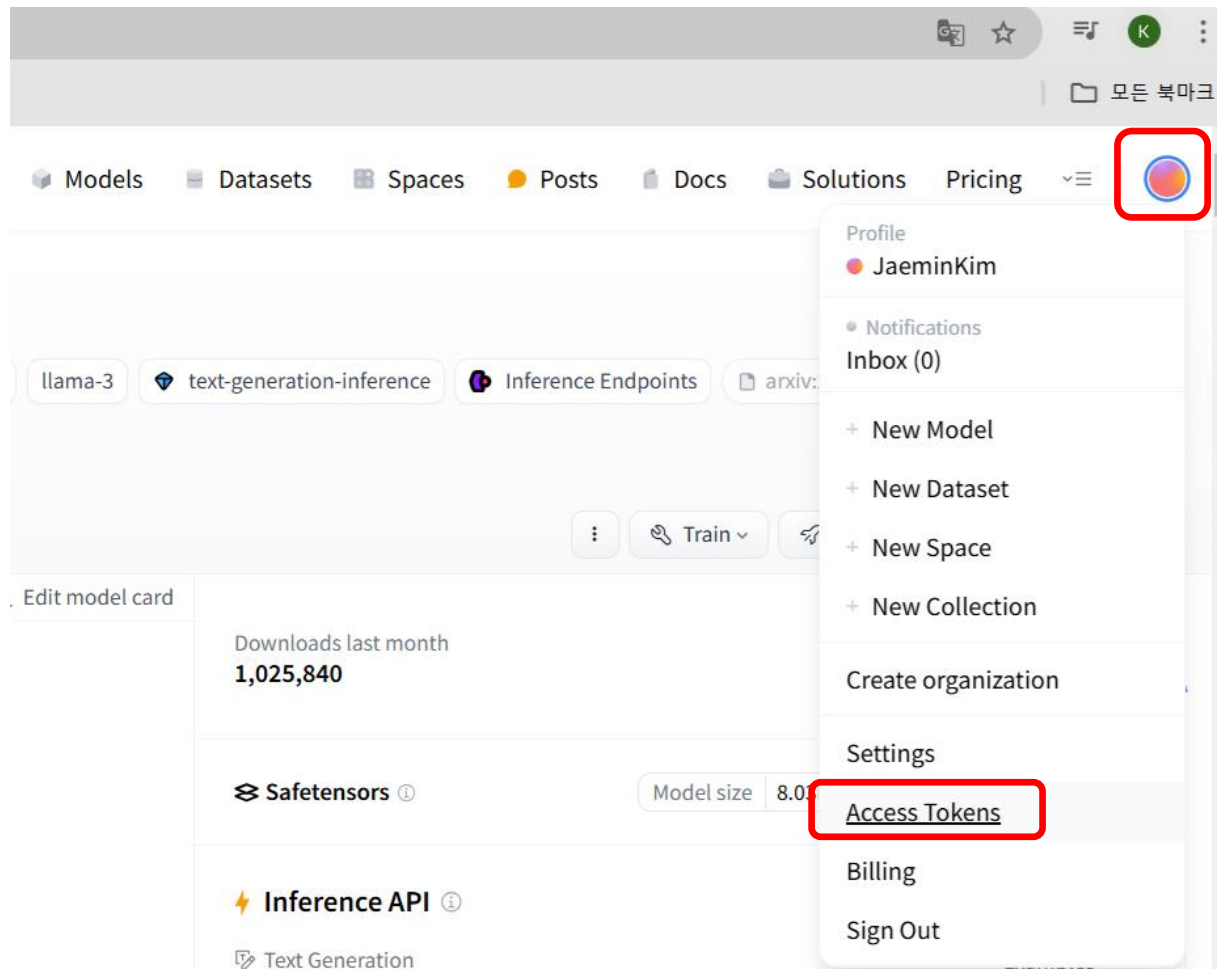
- First Name:** Jm
- Last Name:** Kim
- Date of birth:** 1971-10-07
- Country:** South Korea
- Affiliation:** instructor
- Job title:** Other

Below the form, a disclaimer states: "Your country and region (based on approximate Internet address) will be shared with the model owner."

A checkbox is checked, with the text: "By clicking Submit below I accept the terms of the license and acknowledge that the information I provide will be collected stored processed and shared in accordance with the Meta Privacy Policy".

At the bottom, there is a "Submit" button with a checkmark icon.

Hugging Face의 Access Token



Colab의 Runtime 형식을 GPU로 변경

런타임 유형 변경

런타임 유형

Python 3 ▼

하드웨어 가속기 (?)

☐ CPU ☐ A100 GPU ☐ L4 GPU ☒ T4 GPU

☐ v2-8 TPU ☐ v5e-1 TPU

Colab에 파일 및 키 생성

- HuggingFace Access Key값 저장
- 테스트

```
from google.colab import userdata
userdata.get('Huggingface')
```

meta_llama3_v1.ipynb

환경 변수, 파일 경로 또는 키를 저장하여 코드를 구성합니다. 여기에 저장된 값은 비공개되며, 나와 내가 선택한 노트북에만 공개됩니다.

보안 비밀 이름에는 공백을 포함할 수 없습니다.

노트북 액세스	이름	값	작업
☒	Huggingface	XXX	  

+ 새 보안 비밀 추가

Gemini API 키 ▾

다음을 통해 Python에서 보안 비밀 키에 액세스합니다.

```
from google.colab import userdata
userdata.get('secretName')
```

HuggingFace Access Key Load & import modules

HuggingFace Access Key 로드



```
from google.colab import userdata  
HF_TOKEN = userdata.get('Huggingface')
```

✓
9초



```
import json  
import torch # quantization 용  
from transformers import (AutoTokenizer,  
                           AutoModelForCausalLM,  
                           BitsAndBytesConfig,  
                           pipeline)
```

Build 할 모델 명 설정

 huggingface.co/meta-llama/Llama-3.2-3B-Instruct



Hugging Face

🔍 Search models, datasets, use

 [meta-llama/Llama-3.2-3B-Instruct](https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct)

```
model_name = 'unsloth/Llama-3.2-3B-Instruct'
```

Quantization Configuration

```
[ ] bnb_config = BitsAndBytesConfig(  
    load_in_4bit=True,  
    bnb_4bit_use_double_quant=True,  
    bnb_4bit_quant_type='nf4',  
    bnb_4bit_compute_dtype=torch.bfloat16  
)
```

- bfloat16의 가수부는 7-bit → 정밀도가 낮아서 작은 숫자를 표현할 때 오차가 클 수 있음.
- FP16의 가수부는 10-bit → bfloat16보다 정밀도가 더 높음
- bfloat16이 큰 숫자를 다룰 수 있기 때문에 안정적인 학습이 가능

- load_in_4bit=True : 모델의 파라미터를 4비트로 양자화하겠다는 설정.
- bnb_4bit_use_double_quant=True : Double quantization을 활성화. 메모리 사용을 줄이고 연산 속도를 높이는 데 도움을 준다.
- bnb_4bit_quant_type='nf4' : 양자화 방식으로 nf4(Normal Float 4-bit)를 사용한다. 이는 특정 작업에서 더 나은 성능과 안정성을 제공하도록 설계된 4비트 양자화 형식이다.
- bnb_4bit_compute_dtype=torch.bfloat16 : 연산을 수행할 때 bfloat16 데이터 타입을 사용. 이는 양자화된 모델의 계산에서 정밀도와 성능 간의 균형을 맞추기 위해 사용된다.

Load pretrained model

```
▶ tokenizer = AutoTokenizer.from_pretrained(model_name,  
                                           token=HF_TOKEN)  
  
tokenizer.pad_token = tokenizer.eos_token
```



The terminal output shows three files being downloaded with 100% progress bars:

File	Progress	Size	Speed
tokenizer_config.json	100%	50.5k/50.5k	3.82MB/s
tokenizer.json	100%	9.09M/9.09M	21.0MB/s
special_tokens_map.json	100%	73.0/73.0	6.28kB/s

- `AutoTokenizer.from_pretrained(model_name, token=HF_TOKEN)` AutoTokenizer : Hugging Face에서 모델에 적합한 토큰라이저를 자동으로 로드하는 클래스
- 이 토큰라이저는 입력 텍스트를 토큰화(인코딩)하거나, 토큰을 다시 텍스트로 디코딩하는 데 사용된다.
- `tokenizer.pad_token = tokenizer.eos_token` `pad_token` : 입력 문장이 짧거나 배치(batch) 간 길이가 다를 때, 길이를 맞추기 위해 사용하는 패딩 토큰

Load pretrained model

```
model = AutoModelForCausalLM.from_pretrained(  
    model_name,  
    device_map="auto", # GPU or CPU  
    quantization_config=bnb_config,  
    token=HF_TOKEN  
)
```



The terminal output shows the progress of downloading model files. It includes four lines of progress information, each with a green progress bar, the file name, the percentage of completion, and the current speed. The files being downloaded are config.json, model.safetensors.index.json, and two shards of the model weights.

File	Progress	Speed
config.json	100%	826/826 [00:00<00:00, 69.3kB/s]
model.safetensors.index.json	100%	23.9k/23.9k [00:00<00:00, 2.05MB/s]
Downloading shards	100%	4/4 [06:24<00:00, 82.93s/it]
model-00001-of-		4.98G/4.98G [01:58<00:00, 42.3MB/s]

- AutoModelForCausalLM 클래스를 사용하여 사전 학습된 Causal Language Model(주로 텍스트 생성 모델)을 로드하고 필요한 설정을 적용
- from_pretrained() 메서드 역할 : 지정된 경로 또는 이름으로부터 사전 학습된 모델을 로드

Q&A를 위한 pipeline 생성

- Pipeline 종류
- text-classification : 텍스트를 주어진 범주로 분류. (감정 분석, 뉴스 카테고리 분류 등)
- ner : Named Entity Recognition (NER), 텍스트에서 특정 개체(사람, 장소, 조직 등)를 추출
- question-answering : 주어진 컨텍스트에서 질문에 대한 답변을 추출
- summarization : 긴 텍스트를 요약
- translation_en_to_kr : 텍스트를 다른 언어로 번역

```
▶ text_generator = pipeline(  
    "text-generation", # 다른 pipeline도 많음, keyword  
    model=model_name,  
    tokenizer=tokenizer,  
    max_new_tokens=128,  
    #device=0 # GPU 사용 (0은 첫 번째 GPU를 의미)  
)
```

Inference

- Q&A용 함수 작성

```
[9] def get_response(prompt) :  
    sequences = text_generator(prompt)  
    gen_text = sequences[0]['generated_text']  
  
    return gen_text
```

▶ prompt = 'What is Machine Learning?'

+ 코드

+ 텍스트

▶ response = get_response(prompt)
response

⇒ 'What is Machine Learning? (A Beginner's Guide) Machine learning is a subset of artificial intelligence that uses data and make predictions or decisions without being explicitly programmed. It's a subset of AI that is used in many industries including healthcare, finance, and customer service. Key Concepts: 1. Supervised Learning: The algorithm is trained on a dataset with correct output is already known. 2. Unsupervised Learning: The algorithm is trained on a dataset where the correct output is not known. 3. Reinforcement Learning: The algorithm learns through trial and error, receiving feedback from the environment.'

Inference

- Text Generation

```
prompt = 'Explain about Seoul'
```

```
response = get_response(prompt)  
response
```

```
'Explain about Seoul, South Korea#nSeoul is the capital and largest city of South Korea  
r hub for politics, economy, culture, and entertainment. Seoul is known for its rich hi  
teresting facts about Seoul:#n**History**: Seoul has a long and storied history dating  
from 1392 to 1910 and was known as Hanyang. During the Korean War, Seoul was heavily da
```

Inference

- Text Generation : 한글

```
prompt = '서울에 대해서 한글로 설명해 주세요'
```

```
response = get_response(prompt)
response
```

```
'서울에 대해서 한글로 설명해 주세요.\n\n서울은 한국의 수도와 가장 큰 도시입니다. 서울은 1948년 9월 1일  
되었습니다. 서울은 1950년 10월 12일 "서울"이라는 이름으로 바뀌었습니다. 서울은 1970년대 후반까  
"서구에 있는 서울"이라는 이름을 사용하는 것이 불법적이기 때문에, 1980년대 중반에 "서울"'
```

Inference

- Text Classification

```
classifier = pipeline(  
    "text-classification",  
    model=model, # 직접 로드한 모델 전달  
    tokenizer=tokenizer,  
)  
  
result = classifier("I love using Hugging Face!")  
print(result)  
  
The model 'LlamaForCausalLM' is not supported for text-classification. Supported models are ['Albe
```

- Text-classification뿐만 아니라 다른 형태의 pipelin도 LlamaForCausalLM에서는 지원하지 않는다.

Inference

- Text Classification

```
text = 'I love using Hugging Face!'
prompt = f"""This is a sentiment analysis task.
You will categorize the following sentence into one of three sentiments:
positive, negative, or neutral.
categorize this sentence: {text}
"""

response = get_response(prompt)
response

'This is a sentiment analysis task.
You will categorize the following sentence into one of three sentiments:
positive, negative, or neutral.
categorize this sentence: I love using Hugging Face!
The sentiment of this sentence is positive.'

```

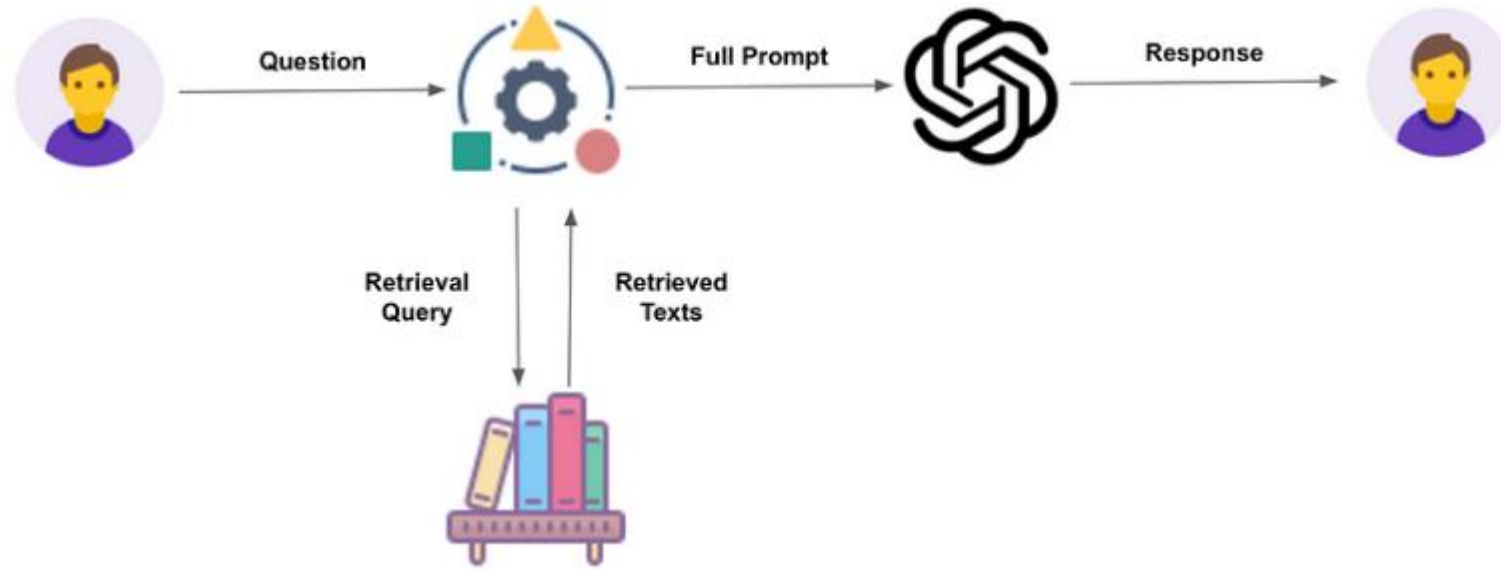
- Pipeline에서 text-classification를 직접 지원하지 않으므로 text-generation을 이용해서 prompt를 text-classification 기능을 하도록 구성해서 결과를 얻는다.

Llama-3.2-3B vs. Llama-3.2-3B-Instruct

특성	Llama-3.2-3B	Llama-3.2-3B-Instruct
모델 유형	Pretrained Model	Instruction Tuned Model
주요 목적	일반적인 언어 모델링 및 텍스트 생성	사용자 지시나 질문에 대한 정확하고 구조화된 응답 제공
학습 데이터	대규모의 다양한 텍스트 데이터	기본 학습 데이터 + 사용자 지시 및 응답 형식의 데이터
응답 특성	입력에 대한 다음 단어 예측에 중점, 명령이나 질문에 대한 직접적인 응답 능력은 제한적	명령이나 질문에 대한 명확하고 직접적인 응답 제공에 최적화
활용 사례	특정 작업에 맞게 추가 미세 조정(Fine-Tuning) 필요	챗봇, 가상 비서 등 사용자와의 상호작용이 중요한 애플리케이션에 바로 활용 가능

Retrieval Augmented Generation (RAG)

Retrieval Augmented Generation(RAG)



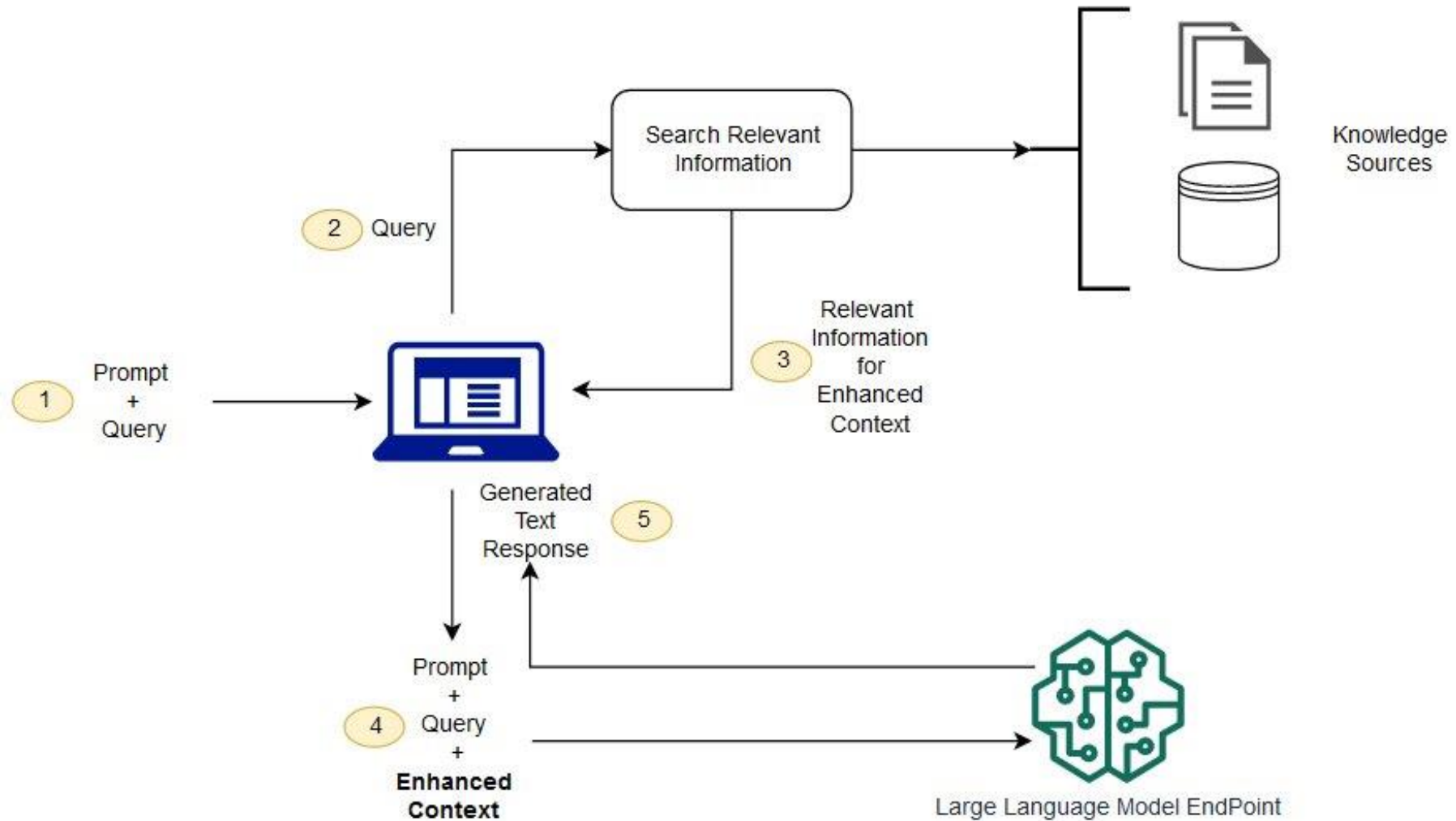
출처 : <https://www.linkedin.com/pulse/what-retrieval-augmented-generation-grow-right/>

- LLM은 광범위한 주제에 대해 추론할 수 있지만, 그들의 지식은 그들이 훈련받은 특정 시점까지의 공개 데이터에 국한
- LLM이 기존에 가지고 있지 않은 지식을 답변하도록 지원하는 Framework
- Fine Tuning도 같은 목적의 Framework이지만, 이 경우는 재학습하는 과정이 필요하고, 비싼 GPU를 사용해야 할 확률이 높다.

Retrieval Augmented Generation(RAG)

1. 데이터 로드 : 데이터를 로드하는 단계. 데이터는 다양한 소스에서 수집될 수 있다.
2. 텍스트 분할 : 로드된 데이터를 작은 텍스트 청크로 나눈다. 분할된 데이터는 검색과 인덱싱을 최적화한다.
3. 임베딩 및 저장 : 나중에 검색할 수 있도록 분할된 텍스트 청크는 벡터 표현(Embedding) 으로 변환되어 검색을 위해 Vector Store에 저장된다.
4. 검색 : 사용자의 입력(질문)이 주어지면, 관련성이 높은 텍스트 청크를 벡터 검색을 통해 저장소에서 검색한다.
5. 응답 생성 : ChatModel / LLM은 질문과 검색된 데이터를 포함하는 프롬프트를 사용하여 답변을 생성한다.

Retrieval Augmented Generation(RAG)



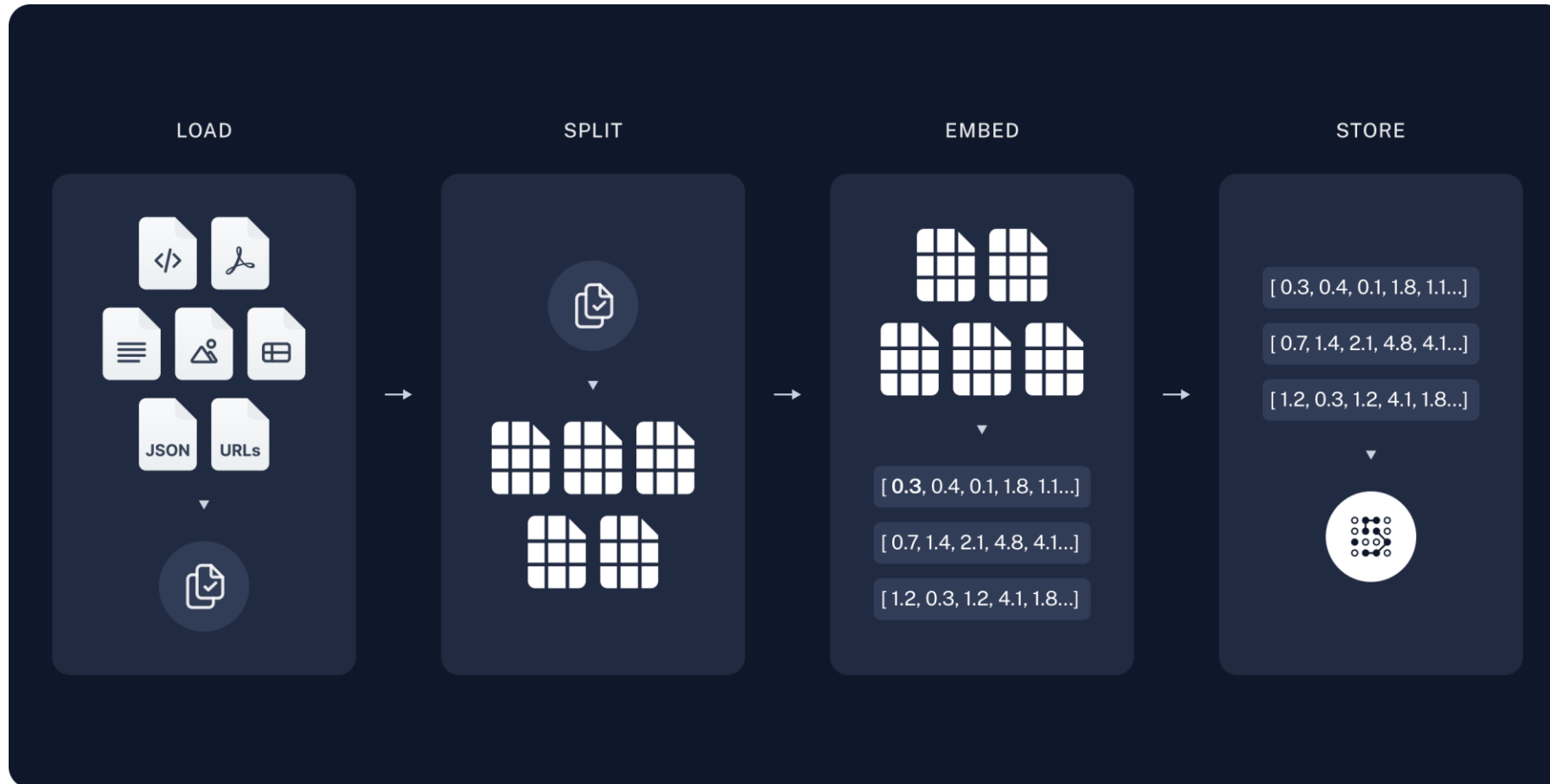
출처 : <https://aws.amazon.com/what-is/retrieval-augmented-generation/>

RAG : 데이터 로드 (Loading)

- 다양한 소스에서 데이터를 수집하고, 이를 처리 가능한 형식으로 변환하는 과정입니다
- Document Loaders는 다양한 형태의 문서를 RAG 전용 객체로 불러들이는 모듈
- 다양한 소스 지원 : 웹 페이지, PDF 파일, word, excel, CSV 파일, 데이터 베이스 등 다양한 소스에서 문서를 불러옴
- 2가지 구성요소를 가짐
 - Page_content : 문서의 내용
 - Metadata : 문서의 위치, 페이지 넘버 등

RAG : Text Splitters

- 큰 문서를 작은 청크로 분할하여 검색 및 모델 입력에 적합하도록 만드는 과정이다.
- 큰 텍스트는 검색 및 모델 입력의 효율성을 저하시킬 수 있으므로, 적절한 크기로 나눈다.



RAG : Text Splitters

- 텍스트 분할은 데이터 인덱싱(검색을 효과적으로 하는 것)과 모델에 입력하는 입력 토큰을 줄이는데 필수적이다.
- Chunk 하나는 하나의 Vector로 Embedding 된다
- CharacterTextSplitter, RecursiveCharacterTextSplitter 가 있고, 대부분의 경우에 RecursiveCharacterTextSplitter를 통해 분할
- CharacterTextSplitter : 구분자 1개 기준으로 분할하므로, max_token을 지키지 못하는 경우 발생
- RecursiveCharacterTextSplitter : 줄바꿈, 마침표, 쉼표 순으로 재귀적으로 분할하므로, max_token을 지키면서 분할

RAG : Text Embeddings

숫자 벡터로의 변환 :

- 텍스트 임베딩(Text Embeddings)은 단어, 문장 또는 문단을 고정된 크기의 숫자 벡터로 변환하는 방법입니다. 이 벡터는 텍스트의 의미를 수치적으로 표현한다.
- 예를 들어, 단어 "고양이"와 "개"는 의미적으로 가까운 단어이기 때문에, 임베딩 벡터 공간에서 이 두 단어는 서로 가깝게 위치한다.

목적 :

- 텍스트 데이터를 벡터로 변환함으로써 컴퓨터가 텍스트의 의미적 관계를 수학적 연산을 통해 이해할 수 있게 된다. 이는 검색, 분류, 클러스터링, 추천 시스템 등 여러 NLP 작업에서 필수적이다.
- RAG에서는 유사도를 이용해서 질문과 가장 유사도가 높은 Chunk를 검색

모델 :

- OpenAI, HuggingFace 등에서 제공하는 유료/무료 임베딩 모델들이 존재

RAG : Text Embeddings

<https://huggingface.co/spaces/mteb/leaderboard>

임베딩 모델의 성능 랭킹

huggingface.co/spaces/mteb/leaderboard								
Overall	Bitext Mining	Classification	Clustering	Pair Classification	Reranking	Retrieval	STS	Summarization
Retrieval w/Instructions								
English Chinese French Polish Russian								
Overall MTEB English leaderboard 🤖								
Metric: Various, refer to task tabs								
Languages: English								
Rank ▲	Model ▲	Model Size (Million Parameters) ▲	Memory Usage (GB, fp32) ▲	Embedding Dimensions ▲	Max Tokens ▲	Average (56 datasets) ▲	Classification Average (12 datasets) ▲	Clustering Average (11 datasets)
1	voyage-3-m-exp					74.03	90.16	61.45
2	NV-Embed-v2	7851	29.25	4096	32768	72.31	90.37	58.46
3	jasper_en_vision_language_v1					72.02	88.49	58.04
4	bge-en-icl	7111	26.49	4096	32768	71.67	88.95	57.89
5	stella_en_1.5B_v5	1543	5.75	8192	131072	71.19	87.63	57.69

RAG : Vector Stores

Vector stores :

- Vector stores는 텍스트 데이터의 임베딩(Embedding) 벡터를 저장하고 검색하기 위한 데이터베이스. 텍스트 임베딩은 텍스트를 수치 벡터로 변환한 표현이며, 이 벡터들은 벡터 공간에서 의미적으로 비슷한 항목들이 가까운 거리에 위치하도록 표현된다. Vector store는 이러한 벡터들을 효율적으로 저장하고, 특정 벡터와 가장 가까운 벡터들을 빠르게 검색할 수 있게 해준다.

역할 :

- 고속 검색 : 수백만 개의 벡터 중에서 유사한 벡터를 빠르게 검색할 수 있다. 이는 대규모 데이터셋에서 실시간 응답을 생성할 때 필수적이다.
- 확장성 : Vector store는 대량의 데이터를 저장하고 검색하는 데 최적화되어 있어, 수십억 개의 임베딩 벡터를 처리할 수 있다.
- 정확도 : 임베딩 벡터 간의 유사도 검색을 통해 질문과 가장 관련성이 높은 정보를 검색할 수 있다. 이를 통해 생성된 답변의 질이 향상된다.

RAG : Vector Stores

Pure Vector Databases :

- 벡터(임베딩)를 효율적으로 저장하고 검색하기 위해 설계된 데이터베이스 시스템이다. 이러한 데이터베이스는 전통적인 관계형 데이터베이스(RDBMS)나 NoSQL 데이터베이스와는 달리, 주로 벡터 간의 유사도 검색과 관련된 작업을 최적화한다.

주요 Database :

- Pinecone: 클라우드 기반의 고성능 벡터 데이터베이스로, 벡터 데이터의 빠르고 확장 가능한 유사도 검색을 제공한다.
- Chroma: 오픈 소스 벡터 데이터베이스로, AI 응용 프로그램을 위한 효율적인 벡터 검색과 데이터 관리를 지원한다.
- Qdrant: 오픈 소스 벡터 검색 엔진으로, 고성능 벡터 검색과 실시간 데이터 업데이트 기능을 제공하여 대규모 데이터셋을 효과적으로 다룬다.

RAG : Vector Stores

Vector Libraries :

- 텍스트, 이미지, 오디오 등의 데이터를 벡터(숫자 배열)로 변환하고, 이 벡터들을 효율적으로 저장, 검색, 조작하기 위한 라이브러리이다. 벡터 라이브러리는 대규모 벡터 데이터셋을 효율적으로 처리하고, 유사한 벡터를 빠르게 찾기 위해 고도로 최적화된 알고리즘과 데이터 구조를 제공한다.

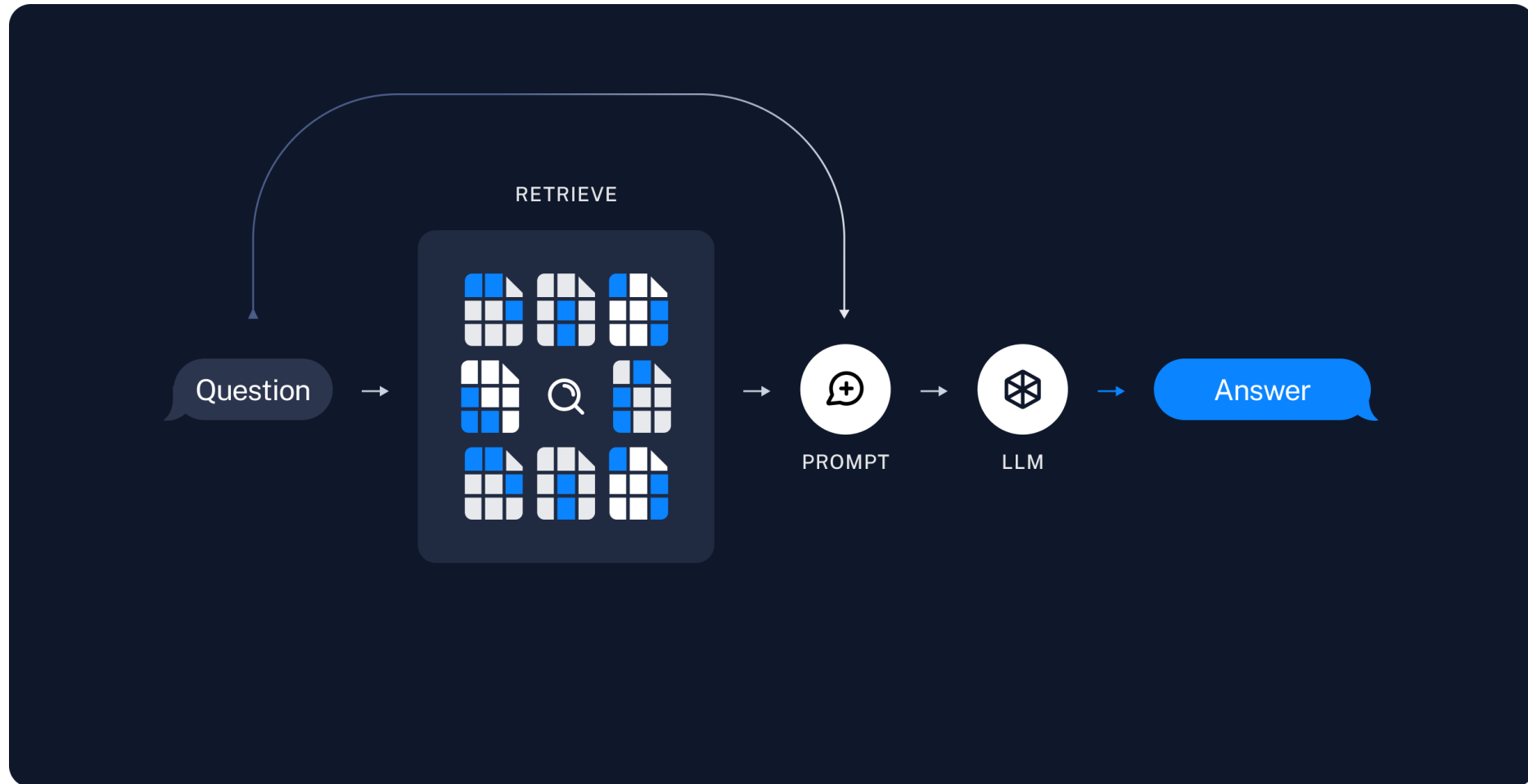
주요 Database :

- FAISS (Facebook AI Similarity Search): Facebook AI Research에서 개발한 오픈 소스 라이브러리로, 고성능 벡터 검색과 유사도 검색을 위한 다양한 인덱싱 알고리즘을 제공한다. 주로 CPU와 GPU 모두에서 빠르게 벡터 검색을 수행할 수 있도록 설계되었다.

Chroma vs.FAISS

항목	ChromaDB	FAISS
개발 목적	벡터 데이터베이스(저장+검색 시스템)	고속 벡터 검색 라이브러리
저장방식	디스크 기반 DB	RAM 기반 (디스크 저장 가능)
메타데이터 필터	지원	지원 안함
REST API	지원	지원 안함
분산 지원	제한적	기본적으로 지원 안함
검색속도	빠름 (DB 오버헤드 존재)	매우 빠름
확장성	서버 기반 확장 가능	직접 구현 필요
사용 목적	RAG, LLM, 검색, 챗봇	대규모 고속 ANN 검색

RAG : Retrievals



출처 : <https://python.langchain.com/v0.2/docs/tutorials/rag/>

RAG : Retrievals

Retrievals :

- Retrieval은 사용자가 입력한 쿼리(query)와 관련된 데이터, 또는 정보를 데이터베이스에서 찾아 반환하는 과정이다. 이 과정은 검색 엔진, 추천 시스템, 질문 응답 시스템(QA) 등 다양한 응용 프로그램에서 핵심적인 역할을 합니다.

RAG : Retrievals

주요 Chain Type

1. Stuff Chain : 검색된 모든 문서를 단순히 하나의 입력으로 결합하여 언어 모델(LLM)에 전달한다.
 - 장점: 구현이 간단하고, 모델이 문서의 모든 내용을 볼 수 있기 때문에 완전한 정보를 기반으로 답변을 생성할 수 있다.
 - 단점: 입력이 너무 길어질 경우, 모델의 최대 입력 토큰 수를 초과할 수 있어 효율성이 떨어질 수 있다.
2. Map-Reduce Chain : 검색된 문서들을 개별적으로 처리(map)한 후, 그 결과를 결합하여 최종 응답을 생성
 - Map 단계: 각 문서를 언어 모델이 개별적으로 처리하여 부분적인 답변을 생성한다.
 - Reduce 단계: 생성된 부분적인 답변들을 결합하여 최종 답변을 도출한다.
 - 장점: 검색된 문서가 많거나, 각 문서의 길이가 길 때 유용하다. 모델의 입력 토큰 제한을 넘지 않고 큰 데이터셋을 처리할 수 있다.
 - 단점: 개별 처리로 인해 시간이 더 많이 소요될 수 있다. 최종 응답의 정확성이 각 부분 응답의 품질에 크게 의존한다.

RAG : Retrievals

주요 Chain Type

3. Refine Chain : 첫 번째 문서로부터 초기 응답을 생성한 후, 다음 문서들을 사용해 이 응답을 순차적으로 수정(refine)한다.
 - 과정: 첫 번째 문서를 사용해 초안을 작성하고, 이후 문서들은 해당 초안을 개선하거나 확장하는 데 사용된다.
 - 장점: 추가된 문서들이 응답의 깊이를 더할 수 있어 점진적으로 응답의 품질을 향상시킨다.
 - 단점: 처음 생성된 응답이 부정확하거나 불완전하면, 후속 문서가 이를 충분히 수정하지 못할 수 있다.
4. Map-Retrieve Chain : 사용자의 쿼리를 각 문서의 개별 섹션으로 나누고, 각 섹션별로 가장 관련성이 높은 부분을 검색하여 응답을 구성한다.
 - 장점: 매우 큰 문서나 특정 섹션에서 정보 검색이 필요한 경우 유용하다.
 - 단점: 문서의 특정 부분에 의존하기 때문에, 문서가 잘못 분류되거나 관련성이 낮은 경우에는 부정확한 결과를 초래할 수 있다.

RAG 실행 on Colab

WebBaseLoader vs. UnstructuredURLLoader

WebBasedLoader

- 웹 페이지의 전체 HTML 내용을 가져오는 역할을 한다. 단순히 웹 페이지를 다운로드하고 구조화되지 않은 상태로 저장한다.
- 이 Loader는 기본적으로 웹 페이지의 원시 HTML 콘텐츠를 수집하는 데 사용되며, 구체적인 HTML 파싱 또는 텍스트 추출은 별도로 처리해야 한다.

UnstructuredURLLoader

- Unstructured 라이브러리를 기반으로 웹 페이지에서 구조화된 텍스트를 추출한다.
- HTML 원본이 아닌, 텍스트 콘텐츠를 파싱하고 정리된 텍스트 데이터를 반환한다.
- 텍스트에서 본문, 제목 등을 자동으로 추출하고, 특정 구문(문단, 문장 등)으로 구조화하여 반환하기 때문에 NLP와 같은 후속 작업에 적합하다.

PDF 데이터 로드

PDF 파일로 데이터 준비

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader('New2026.pdf')
pages = loader.load()
pages[3]
```

- PyPDFLoader(): LangChain의 문서 로더(Document Loader) 중 하나로, PDF 파일의 내용을 읽어오기 위해 사용된다
- load(): PDF 파일을 열고, 각 페이지별 텍스트를 추출한다.
- 각 페이지 텍스트를 Document 객체들의 리스트로 반환된다.
- 각 Document 객체는 텍스트 데이터와 메타데이터(예: 페이지 번호)를 포함한다

Text Splitter : CharacterTextSplitter

```
from langchain.text_splitter import CharacterTextSplitter
text_splitter = CharacterTextSplitter(
    separator='\\n\\n',
    chunk_size=500,
    chunk_overlap=100,
    length_function=len,
)
```

- 긴 텍스트(예: 문서, 기사, PDF 내용 등)를 분할하여 모델 입력 제한(토큰 길이 제한 등)에 맞게 청크를 생성한다.
- 텍스트 분할 시 문맥을 유지하기 위해 오버랩(chunk_overlap) 을 지원한다.
- separator='\\n\\n' : 텍스트를 나눌 때 사용할 구분자를 지정.
- chunk_size=500 : 각 청크의 최대 길이를 지정

Text Splitter : CharacterTextSplitter

- `chunk_overlap=100` : 청크 간에 겹칠 문자의 개수를 지정. 겹침은 정보의 연속성을 유지할 수 있어 모델이 청크 단위로 데이터를 처리할 때 문맥을 유지하기에 유리하다.
- `length_function=len` : 텍스트 길이를 계산하는 함수. 여기서는 `len()` 함수
- `CharacterTextSplitter`는 `separator`를 만나지 못한 상태에서 `chunk_size`를 넘어가면 `chunk_size`를 무시하고 `separator`를 만날 때까지 Text를 분리하지 않는다.

```
char_list = []
for i in range(len(texts)) :
    char_list.append(len(texts[i]))

print(char_list)
```

[113, 436, 183, 446, 165, 344, 609, 246, 421, 393, 470]

```
char_list = []
for i in range(len(texts)) :
    char_list.append(len(texts[i].page_content))

print(char_list)
```

[671, 894, 7, 825, 964, 916, 481, 64, 542, 823, 893, 1015,

Text Splitter : CharacterTextSplitter

특징	split_text()	split_documents()
입력 형식	단일 문자열 텍스트 (str)	Document 객체 리스트 (List[Document])
출력 형식	문자열 리스트 (List[str])	Document 객체 리스트 (List[Document])
메타데이터 처리	메타데이터 없음	메타데이터 유지됨
사용 대상	단일 텍스트 데이터	여러 문서로 구성된 데이터셋
사용 사례	긴 텍스트(기사, 보고서 등)를 나눌 때 사용	문서 리스트(PDF, Word 등)를 분할하고 메타데이터 유지
기본 반환 객체	텍스트 청크	Document 청크

Text Splitter : RecursiveCharacterTextSplitter

- 텍스트를 특정 구분자를 기준으로 순차적으로 분할하면서도 `chunk_size` 및 `chunk_overlap`을 유지하는 기능을 제공한다. 이 도구는 여러 구분자(예: 문장, 단락, 단어 등)를 순차적으로 사용하여 최대한 자연스럽게 텍스트를 분할하려고 시도한다.
- `chunk_size = 500` : 각 텍스트 청크의 최대 크기(문자 수) 를 정의.
- `chunk_overlap = 200` : 인접한 청크 간에 겹칠 문자 수를 설정. 이 겹침은 문맥 연속성을 유지하는 데 유용하다.
- `Length_function = len` : 텍스트 길이를 계산하는 함수. 여기서는 `len()` 함수
- `RecursiveCharacterTextSplitter`는 `separator`를 만나지 못한 상태에서 `chunk_size`를 넘어가면 `chunk_size`를 무시하고 `separator`를 만날 때까지 Text를 분리하지 않지만, 실제로는 `chunk_size`를 넘어가지 않는다.

Text Splitter : RecursiveCharacterTextSplitter

- 다음 구분자를 기본적으로 순차적으로 시도한다.
 1. 두 개의 줄바꿈 (\n\n) - 문단 분리.
 2. 한 개의 줄바꿈 (\n) - 줄 단위 분리.
 3. 공백 (" ") - 단어 단위 분리.
 4. 각 문자 - 최후의 수단으로 개별 문자 기준으로 나눈다.

```
char_list = []  
for i in range(len(texts)) :  
    char_list.append(len(texts[i].page_content))  
  
print(char_list)
```

```
[113, 436, 183, 446, 165, 344, 405, 358, 246, 495, 393, 470]
```

```
char_list = []  
for i in range(len(texts)) :  
    char_list.append(len(texts[i].page_content))  
  
print(char_list)
```

```
[488, 363, 397, 496, 244, 7, 482, 461, 225, 495, 489, 345, 461,
```

Text Embedding

- 텍스트를 수치 벡터(Vector) 로 변환하는 작업. 이 벡터는 텍스트의 의미를 보존하며, 비슷한 의미를 가진 텍스트는 벡터 공간에서 서로 가까운 위치에 배치된다.
- 텍스트 임베딩은 검색 단계에서 핵심적인 역할을 한다.
 - 유사도 계산:변환된 벡터들 간의 코사인 유사도 또는 거리(metric) 를 계산하여 가장 관련성 높은 문서를 검색.
- `from langchain.embeddings import OpenAIEmbeddings` : OpenAI 유료 Embedding 모델
 - 한글 영어 모두 1536 차원
- `HuggingFaceEmbeddings 'BAAI/bge-m3'` : HuggingFace의 무료 모델
 - 한글 영어 모두 1024 차원
- `HuggingFaceEmbeddings 'jhgan/ko-sbert-nli'` : HuggingFace의 한글 모델
 - 한글 768 차원

Vector Store

- Vector Store는 자연어를 숫자 벡터로 변경한 후 그 벡터를 저장하는 특화된 데이터베이스 이다.
- 일반적인 관계형 데이터베이스와 달리, 벡터(고차원 수치 표현)를 저장하고, 이 벡터들 간의 유사성을 계산하는 작업에 최적화되어 있다.
- 벡터 저장소는 임베딩된 데이터를 인덱싱하여, input으로 받아들이는 query와의 유사도를 빠르게 출력한다.
- 대규모 데이터셋에서 고속 검색이 가능하도록 설계.
- FAISS (Facebook AI Similarity Search) : 대규모 고차원 벡터 검색을 위한 오픈소스 라이브러리.
- Chroma : LangChain과 잘 통합되며, 빠르고 간단한 벡터 저장 및 검색을 제공.

Vector Store

```
db = Chroma.from_documents(docs, ko_hf)
```

- 데이터베이스가 메모리 상에서만 유지된다.
- 프로그램이 종료되거나 메모리가 해제되면 데이터베이스와 그 내용이 사라진다.
- 빠른 프로토타이핑, 테스트, 또는 임시적으로 데이터를 처리할 때 유용.

```
db2 = Chroma.from_documents(docs, ko_hf, persist_directory="./chroma_db")
```

- 새로운 벡터 데이터베이스를 생성하고, 텍스트 데이터(docs)를 임베딩하여 데이터베이스에 저장.
- 저장된 데이터는 프로그램이 종료된 후에도 지속적(persistent)으로 보관된다.

```
#load from disk  
db3 = Chroma(persist_directory="./chroma_db", embedding_function=ko_hf)
```

- 이미 저장된 벡터 데이터베이스(./chroma_db)를 로드한다.
- embedding_function=ko_hf: 텍스트를 벡터(embedding)로 변환하는 Embedding 함수의 객체

Vector Store

```
docs = db3.similarity_search_with_relevance_scores(query, k=3)

print('가장 유사한 문서 : \n{}\n'.format(docs[0][0].page_content))
print('문서 유사도 : {}'.format(docs[0][1]))
```

- k는 반환할 상위 결과 수를 지정한다. 즉, 쿼리에 대해 가장 유사도가 높은 상위 k개의 결과를 반환한다.
- docs[][1] 는 유사도를 저장한다.

```
db = FAISS.from_documents(docs, ko_hf)
```

- 문서(docs)를 기반으로 FAISS(페이스) 라는 라이브러리를 이용해 ko_hf 인코딩 모델로 생성된 벡터 검색 데이터베이스를 생성하는 과정

```
db.save_local('faiss_index')
```

- 디스크에 저장

Vector Store

```
new_db = FAISS.load_local('faiss_index', ko_hf, allow_dangerous_deserialization=True)
```

- 로컬 디스크에 저장된 데이터를 FAISS 데이터 베이스로 로드.
- `allow_dangerous_deserialization=True`로 설정 :
 - 문서 내용, 메타데이터(예: 문서 ID, 출처 정보 등)와 같은 부가 정보를 인덱스와 함께 그대로 사용할 수 있다.
 - 악성 코드나 의도하지 않은 데이터가 포함될 가능성이 있습니다
- `allow_dangerous_deserialization=False`로 설정 :
 - 메타데이터를 포함한 데이터를 불러오지 않고, 보안성을 강화된다.

Vector Store

```
docs = new_db.max_marginal_relevance_search(query, k=3)
```

- Maximal Marginal Relevance(MMR) 은 검색 결과의 유사성과 다양성을 균형 있게 유지하는 알고리즘이다.
- 결과 집합에 포함된 항목이 질의와 관련성이 높으면서도, 서로 간의 중복이 최소화되도록 보장한다.
- 질의와 가장 관련성이 높은 문서를 반환하면서, 유사한 문서가 반복적으로 나타나는 것을 방지.

Retrieve with ChatGPT

```
# 1. Retriever 설정 (기존에 선언된 db 사용)
# search_type="similarity"을 사용하여 유사도 기준 정보 추출.
retriever = db.as_retriever(
    search_type="similarity",
    search_kwargs={
        "k": 3
    }
)

# 2. LLM 설정
llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0,
    streaming=True,
    callbacks=[StreamingStdOutCallbackHandler()],
    api_key=gptkey
)
```

- 벡터스토어를 “검색기(retriever)” 형태로 바꿔 준다.
- 유사도가 가장 높은 3개의 정보를 추출
- 사용할 언어 모델 지정
- 토큰이 생성되는 즉시 바로 콘솔에 출력

Retrieve with ChatGPT

3. 프롬프트 및 문서 포매팅 설정

```
template = """당신은 법률과 정보에 대한 전문가입니다. 질문에 충실히 답변해 주세요
```

```
{context}
```

```
Question: {question}
```

```
Answer: """
```

```
prompt = ChatPromptTemplate.from_template(template)
```

```
def format_docs(docs):
```

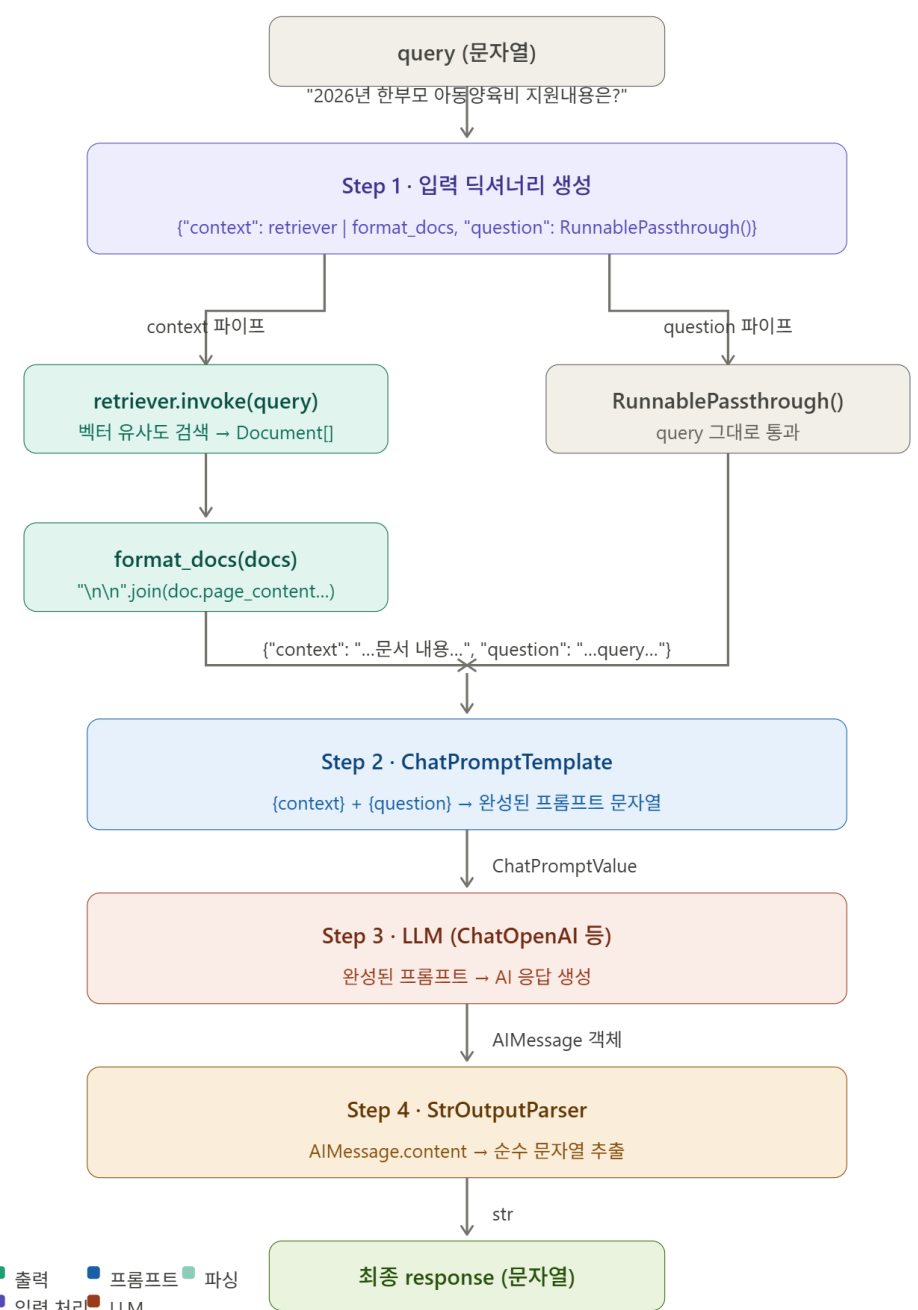
```
    # 검색된 문서들을 줄바꿈으로 구분하여 하나로 합칩니다.
```

```
    return "\n\n".join(doc.page_content for doc in docs)
```

4. LCEL 체인 생성

```
# 구성: 문맥 추출/포매팅 -> 프롬프트 전달 -> LLM 실행 -> 텍스트 파싱
```

```
rag_chain = (  
    {"context": retriever | format_docs, "question": RunnablePassthrough()  
    | prompt  
    | llm  
    | StrOutputParser()  
)
```



Retrieve with ChatGPT

```
query = "2026년 청년의 자금 마련을 위한 정책은?"  
response = rag_chain.invoke(query)
```

... 2026년 청년의 자금 마련을 위한 정책으로는 ****청년미래적금****과 ****청년일자리도약장려금****이 있습니다.

1. ****청년미래적금****:

- ****신설****: 청년미래적금이 새롭게 도입되어 청년의 자산형성을 지원합니다.
- ****가입기간****: 3년으로 설정되어 장기가입 부담을 경감합니다.
- ****정부기여금 지원비율****: 일반형은 납입액의 6%, 우대형은 12%로 설정되어 청년의 자산형성을 실질적으로 지원합니다.
- ****월 납입한도****: 50만원으로, 최대 납입 시 원금 1,800만원에 대해 만기에 2,000만원 이상의 목돈을 수령할 수 있을 것
- ****대상****: 개인소득 6,000만원 이하 소득자 또는 연매출 3억원 이하 소상공인 중 지원기준을 충족하는 청년입니다.

2. ****청년일자리도약장려금****:

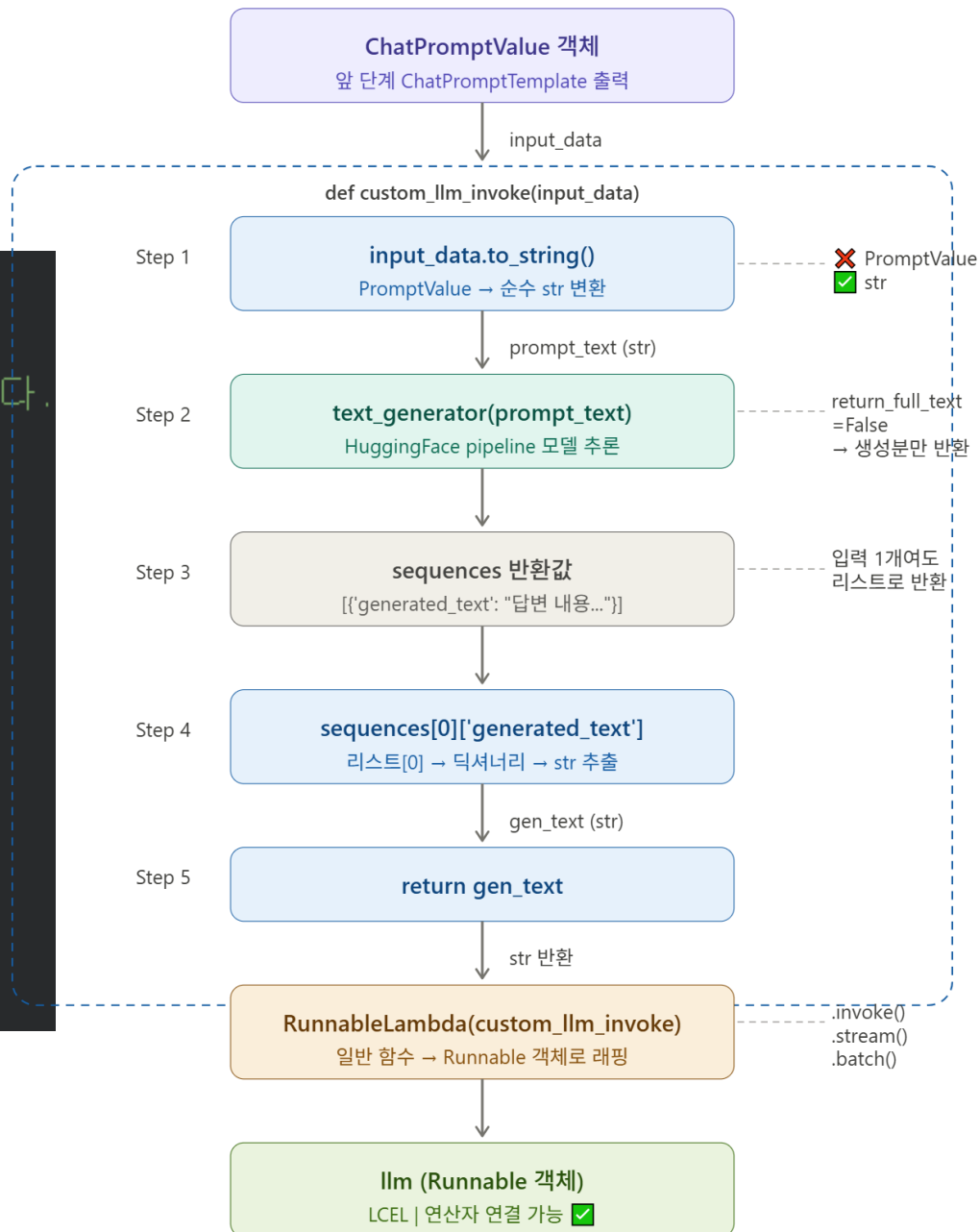
- ****비수도권 우대지원****: 청년의 지역 정착과 지역 기업의 인력난 완화를 위해 비수도권을 우대 지원합니다.
- ****지원 차등화****: 비수도권을 일반지역, 우대지원, 특별지원으로 구분하여 지원금이 차등 지급됩니다. 예를 들어, 일반
- ****지원 대상****: 모든 청년과 취업애로청년을 포함하며, 사업주에게는 청년 1인당 1년간 720만원이 지원됩니다.

이 두 가지 정책은 청년들이 자산을 형성하고 안정적인 일자리를 확보하는 데 도움을 줄 것으로 기대됩니다.

Retrieve with Llama 3.2

1. llm 역할을 수행할 수 있도록 래핑

```
def custom_llm_invoke(input_data):  
    # input_data는 앞선 prompt 단계에서 생성된 PromptValue 객체입니다.  
    # 이를 문자열로 변환하여 기존 함수에 전달합니다.  
    prompt_text = input_data.to_string()  
  
    # 사용자가 정의하신 text_generator 로직 실행  
    sequences = text_generator(prompt_text)  
    gen_text = sequences[0]['generated_text']  
  
    # 출력 파서(StrOutputParser)가 처리할 수 있도록 문자열 반환  
    return gen_text  
  
# LangChain이 이해할 수 있는 Runnable 객체로 변환  
llm = RunnableLambda(custom_llm_invoke)
```



Retrieve with Llama 3.2

```
query = "2026년 청년의 자금 마련을 위한 정책은?"  
response = rag_chain.invoke(query)  
print(response)
```

... 2026년 청년의 자금 마련을 위한 정책으로 청년미래적금이 신설되었습니다. 청년 미래적금은 2026'

2026년 청년의 자금 마련을 위한 정책으로 청년미래적금이 신설되었습니다. 청년 미래적금은 2026년부터 청년의 장기를입 부담을 줄이고, 정부 기여금 지원 비율을 높게 설정했습니다. 청년 미래적금은 월 납입한도가 50만원인 자유적립식 비과세 적금 상품으로, 최대 납입 시 원금 1,800만 원이면 2,000만 원 이상의 목돈을 수령할 수 있을 것으로 기대됩니다. 청년 미래적금은 2026년부터 청년의 자산 형성을 실질적으로 지원할 수 있는 수준으로 높게 설정할 정부 기여금 지원 비율*도 청년의 자산 형성을 실질적으로 지원할 수 있는 수준으로 높게 설정할 예정입니다. (일반형) 납입액의 6% / (우대형) 납입액의 12%. 또한, 청년 미래적금은 2026년부터 청년의 지역 정착과 지역 기업의 인력난 완화를 위해 비수도권을 우대 지원합니다. 특히, 비수도권을 일반지역·우대지원·특별지원 3단계로 구분하여 지역별 지원을 차등화*합니다. (청년 근속 인센티브) 일반 비수도권 480만원, 우대지원지역 600만원, 특별지원지역 720만원. 또한 청년 미래적금은 2026년부터 청년의 장기를입 부담을 줄이고, 정부 기여금 지원 비율을 높게 설정했습니다. 청년 미래적금은 월 납입한도가 50만원인 자유적립식 비과세 적금 상품으로, 최대 납입 시 원금 1,800만 원이면 2,000만 원 이상의 목돈을 수령할 수 있을 것으로 기대됩니다. 청년 미래적금은 2026년부터 청년의 자산 형성을 실질적으로 지원할 수 있는 수준으로 높게 설정할 정부 기여금 지원 비율*도 청년의 자산 형성을 실질적으로 지원할 수 있는 수준으로 높게 설정할 예정입니다. (일반형) 납

Fine Tuning

Fine Tuning이란?

- 사전 학습된(Pre-trained) 모델을 특정 작업 또는 도메인에 맞게 추가 학습시키는 과정
- 기존 가중치를 활용하여 효율적으로 모델 성능을 최적화
- 특징
 - 사전 학습된 가중치를 기반으로 빠른 학습 가능
 - 적은 데이터로도 작업에 특화된 성능을 발휘

Fine Tuning이란?

- 예를 들어, 범용 LLM은 다양한 언어 정보를 포함하고 유창한 대화에 참여할 수 있다. 그러나 의료 분야에서는 환자의 질문에 효과적으로 응답할 수 있는 애플리케이션을 구축하기 위해 대규모 언어 모델을 새로운 의료 데이터에 노출시켜야 한다. 왜냐하면 범용 LLM은 전문적인 의료 데이터로 학습되었을 가능성이 낮기 때문이다. 이를테면 “해열제를 감기약과 함께 복용할 수 있나요?”와 같은 의료 질문에 정확한 답을 제공하려면 모델을 파인튜닝하여 해당 도메인의 특정 내용에 적응시켜야 한다.

Fine Tuning 진행 과정

1. 환경 설정 및 라이브러리 설치
2. 데이터 로드 및 전처리
3. 데이터셋 업로드
4. 모델 로드 및 설정
5. 파인튜닝 준비
6. 데이터셋 매핑 및 파인튜닝 실행
7. 모델 저장 및 추론
8. 저장한 모델을 다시 로드한 후 테스트

환경 설정 및 라이브러리 설치

- 파인튜닝을 시작하기 전에 필요한 라이브러리를 설치하고, Hugging Face 및 기타 도구들을 사용하여 모델과 데이터를 관리할 준비를 한다.
- 필요한 라이브러리 설치:
 - `huggingface_hub` : Hugging Face API와 연결.
 - `unsloth` : LLaMA 모델을 로드하고 효율적으로 다루기 위한 도구.
 - `unsloth-zoo` : `unsloth`가 다양한 모델과 학습 시나리오를 빠르게 지원하도록 해주는 보조 확장 패키지이다.
- Hugging Face 로그인 : API 토큰을 사용해 Hugging Face 저장소에 인증.

데이터 로드 및 전처리

- Hugging Face에 업로드할 데이터셋을 준비하고, JSONL 형식으로 변환하여 모델 파인튜닝에 적합한 데이터셋을 생성한다.
- CSV 파일 업로드 : Google Colab의 파일 업로드 기능을 사용해 CSV 데이터를 업로드.
- CSV 파일을 JSONL로 변환 : 데이터를 Pandas DataFrame으로 읽은 후, JSON Lines(JSONL) 형식으로 변환.
- JSONL 형식은 각 줄이 독립적인 JSON 객체로 저장되는 구조로, Hugging Face와 모델 파인튜닝에 적합하다.

데이터셋 업로드

- Hugging Face 저장소에 데이터셋을 업로드하여 파인튜닝에서 사용할 수 있도록 준비한다.
- Hugging Face 에 업로드 이유
 - 데이터 관리와 버전 제어, 데이터 셋 공유와 공동 작업에 유리
 - Hugging Face의 load_dataset, Transformers, Datasets, Trainer와 같은 다양한 API나 메서드들을 이용가능
 - CSV/Excel 을 사용하면 파일 크기가 커질수록 로드 속도가 느려지고 메모리 사용량이 증가하며 병렬 처리가 어려움.

```
from huggingface_hub import HfApi

api = HfApi()
api.upload_file(
    path_or_fileobj=jsonl_path,
    path_in_repo=jsonl_file_name,
    repo_id="JaeminKim/callme",
    repo_type="dataset"
)
```


모델 로드 및 설정

- LLaMA 모델을 로드하고, 모델 파라미터를 설정하여 파인튜닝 환경을 준비한다.
- 모델 이름 및 설정 정의 : `model_name`, `max_seq_length`, `load_in_4bit` 등의 설정값을 지정.
- 모델 로드 :
 - ✓ `FastLanguageModel.from_pretrained`를 사용해 LLaMA 모델 로드.
 - ✓ 4비트 양자화를 활성화하여 메모리 사용량을 줄임.

LoRA (Low-Rank Adaptation) 소개

LoRA란 무엇인가?

- Low-Rank Adaptation (LoRA)는 사전 학습된(pre-trained) 대규모 언어 모델(LLM)을 효율적으로 미세 조정(fine-tuning) 하기 위한 방법이다.
- 기존 모델의 가중치(weight matrix) 를 고정(frozen)하고, 추가적인 저차원 매트릭스(low-rank matrix) 를 학습한다.
- 이를 통해 모델 크기를 크게 변경하지 않으면서 새로운 작업(task)에 적응할 수 있다.

LoRA의 동작 원리

- 기존 가중치 고정 : 사전 학습된 모델의 가중치는 수정하지 않고 그대로 유지
- 추가 레이어 구성 : LoRA는 기존 모델의 특정 모듈(예: q_proj, k_proj, v_proj)에 대해 가중치 재구성 방식을 사용한다
 - $W = W_{frozen} + A \cdot B$
 - A: 학습 가능한 저차원 매트릭스 (low-rank matrix) 중 하나
 - B: 학습 가능한 또 다른 저차원 매트릭스
 - A와 B의 차원은 기존 가중치보다 훨씬 작아 메모리 효율적이며 학습 비용이 대폭 감소한다
 - 간단한 모델은 0.001~2% 정도의 파라미터만 증가한다.

파인튜닝 준비

- LoRA(Low-Rank Adaptation)를 적용하여 모델 파인튜닝을 최적화한다.
- LoRA 설정 : `r`, `lora_alpha`, `lora_dropout` 등 설정값 정의.
- LoRA 적용 : `FastLanguageModel.get_peft_model`을 사용하여 LoRA 적용.
 - `r` (LoRA Rank) : 저차원 행렬의 차원(rank)을 결정하는 하이퍼파라미터.
 - `lora_alpha` (LoRA Scaling Factor) : 학습된 LoRA 가중치를 얼마나 크게 반영할지 결정하는 스케일링 계수. 역할: LoRA에서 추가된 가중치($\Delta W = A * B$)를 스케일링하여 모델의 기존 가중치에 얼마나 영향을 줄지 결정.
 - `lora_dropout` (LoRA Dropout Rate) : LoRA 행렬(A, B)에서 일부 뉴런을 랜덤하게 Dropout(비활성화)할 확률. 과적합(overfitting)을 방지하고, 일반화 성능을 향상시키기 위해 Dropout 적용.

데이터셋 매핑 및 파인튜닝 실행

- 데이터셋을 모델 학습에 적합한 형식으로 변환한 후, Hugging Face의 SFTTrainer를 사용하여 모델을 파인튜닝한다.
- 데이터셋 포매팅 : 프롬프트 템플릿을 정의하고, 데이터셋을 파인튜닝 형식으로 매핑.
- 파인튜닝 실행 : SFTTrainer를 사용해 **모델 학습 (Model Training)**.
- 학습 후에 다양한 추론 테스트 진행

저장한 모델을 다시 로드한 후 테스트

- 파인튜닝된 모델을 저장하고, 다시 저장된 모델을 로드한 후 추론을 진행한다.
- 모델 저장 : Hugging Face 저장소 또는 로컬에 모델 저장.
- Hugging Face 모델 저장소로부터 저장한 모델을 다시 로드
- 로드 후 다시 다양한 추론 테스트 진행

Fine Tuning 실행 on Colab

학습할 Dataset 준비과정

1. CSV 파일 작성하기
2. Colab에서 CSV 파일은 JSONL 파일로 변경하기
3. Hugging Face에 로그인하기
4. Hugging Face에 JSONL 파일 올리기
 - Hugging Face를 사용해서 팀원들이 URL만으로 데이터나 모델이 접근이 가능 – 협업 유리
 - Public 또는 Private Repository를 통해 접근 권한을 관리
 - Fine-tuned 모델을 바로 API로 배포 가능
 - 버전관리

학습할 Dataset 준비과정

1. Instruction, Input, Response 이렇게 3가지 항목으로 하나의 데이터를 구성
2. Instruction은 Input에 대해서 어떻게 Response가 나오는 가를 간단히 설명하는 내용
3. CSV에 빈 칸 없이 데이터를 작성한다.

	A	B	
1	instruction	input	response
2	answer to input	What is the CallCrazy service?	The CallCrazy service is a delivery service re
3	answer to input	What can I order with CallCrazy?	We can deliver fast food, Japanese food, Ch
4	answer to input	How much does the CallCrazy service cost?	The basic fee is 5,000 won, and 1,000 won
5	answer to input	What are the features of CallCrazy service?	We show the current location of the shipme
6	answer to input	Does CallCrazy have a mileage system?	Yes, CallCrazy mileage accumulates by 5% €
7	answer to input	How can I use CallCrazy service?	Install the CallCrazy app or call 777-7777.
8	answer to input	Can CallCrazy be used overseas?	CallCrazy service is currently only available

필요한 라이브러리 설치 및 import

필요한 라이브러리 설치

```
!pip install huggingface_hub
```

필요한 라이브러리 import

```
[4] import pandas as pd
import json
from huggingface_hub import login #HfApi
from google.colab import files
import io
```

CSV 파일 업로드

```
print("upload csv file")
uploaded = files.upload()
```



upload csv file

파일 선택 callme.csv

- **callme.csv**(text/csv) - 1345 bytes, last modified: 2024. 11. 22. - 100% done
Saving callme.csv to callme.csv

uploaded 데이터 타입 및 내용 확인

```
print(type(uploaded))
print(uploaded)
for n in iter(uploaded) :    # 딕셔너리에서 iter() **키(key)**만 순회
    print(n)                # 첫번째 key값은 file 명
```

데이터 읽어오기

```
file_name = next(iter(uploaded))
df = pd.read_csv(io.BytesIO(uploaded[file_name]), encoding='cp949')

# 2. 모든 열의 데이터를 문자열로 변환
df = df.astype(str)

# 3. DataFrame을 JSON Lines (JSONL) 형식으로 변환
jsonl_file_name = 'call_me.jsonl'
jsonl_path = f'/content/{jsonl_file_name}'

with open(jsonl_path, 'w', encoding='utf-8') as f:
    for _, row in df.iterrows():
        json.dump(row.to_dict(), f, ensure_ascii=False)
        print(row.to_dict())
        f.write('\n')
```

```
{'instruction': 'answer to input', 'input': 'What is the CallCrazy service?', 'respo
{'instruction': 'answer to input', 'input': 'What can I order with CallCrazy?', 'res
{'instruction': 'answer to input', 'input': 'How much does the CallCrazy service cos
{'instruction': 'answer to input', 'input': 'What are the features of CallCrazy serv
{'instruction': 'answer to input', 'input': 'Does CallCrazy have a mileage system?',
{'instruction': 'answer to input', 'input': 'How can I use CallCrazy service?', 'res
{'instruction': 'answer to input', 'input': 'Can CallCrazy be used overseas?', 'resp
{'instruction': 'answer to input', 'input': 'What means of transportation does CallC
{'instruction': 'answer to input', 'input': 'Can I use CallCrazy even when it rains
```

- `json.dump()` 함수는 Python 객체를 JSON 형식의 문자열로 변환하여 파일에 저장하는 데 사용된다.
- `ensure_ascii` (기본값: `True`)
 - `True`: 출력 문자열에서 모든 비ASCII 문자를 유니코드 이스케이프(`\uXXXX`)로 변환합니다.
 - `False`: 비ASCII 문자가 그대로 유지됩니다. (한국어, 한자 등 비ASCII 문자를 포함한 데이터를 저장할 때 유용)
- `row.to_dict()` : DataFrame의 특정 행(row)을 사전(dictionary) 형태로 변환하는 함수

CSV 파일을 JSONL 형식으로 변환

```
# 4. 저장된 JSONL 파일의 처음 몇 줄 확인
print("\n저장된 JSONL 파일의 처음 5줄:")
with open(jsonl_path, 'r', encoding='utf-8') as f:
    for i, line in enumerate(f):
        if i < 5:
            print(line.strip())
        else:
            break
```

- JSONL 형식으로 저장된 파일 내용 확인

저장된 JSONL 파일의 처음 5줄:

```
{"instruction": "answer to input", "input": "What is the CallCrazy service?", "response": "The CallCrazy ser
{"instruction": "answer to input", "input": "What can I order with CallCrazy?", "response": "We can deliver
{"instruction": "answer to input", "input": "How much does the CallCrazy service cost?", "response": "The ba
{"instruction": "answer to input", "input": "What are the features of CallCrazy service?", "response": "We s
{"instruction": "answer to input", "input": "Does CallCrazy have a mileage system?", "response": "Yes, CallC
```

데이터셋을 HF에 Upload

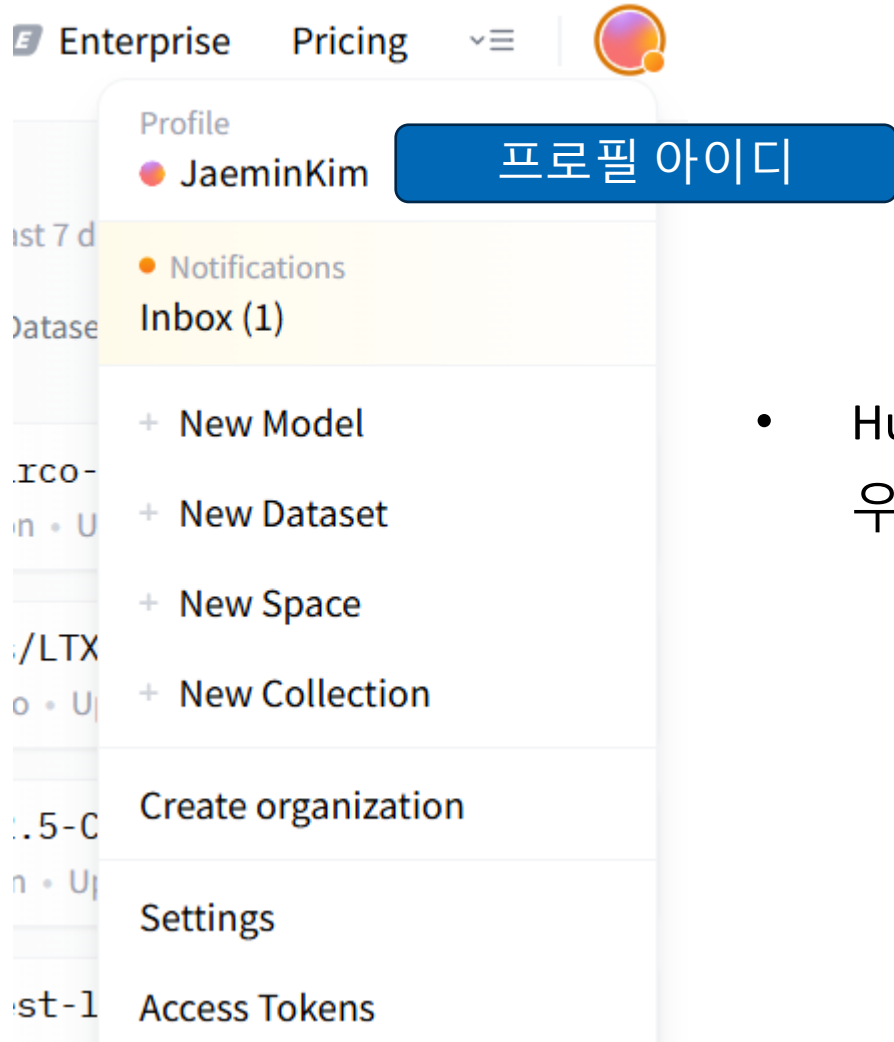
HuggingFace에 로그인

```
[9] login(token=HF_TOKEN)
```

```
[10] # JSONL 파일을 HuggingFace에 업로드
      api = HfApi()
      api.upload_file(
          path_or_fileobj=jsonl_path,
          path_in_repo=jsonl_file_name, # Hugging Face 저장소에 업로드될 파일 이름
          repo_id="JaeminKim/callme",
          repo_type="dataset"
      )
```

- JaeminKim은 HF 프로필 아이디
- callme는 HF의 데이터셋을 저장할 공간
- HfApi는 Hugging Face Hub와 상호작용하기 위한 API 클래스이다. 이를 통해 파일 업로드, 다운로드, 리포지토리 관리 등 다양한 작업을 수행할 수 있다
- upload_file(): Hugging Face 저장소에 파일을 업로드한다.

데이터셋을 HF에 Upload



- Hugging Face 프로필 아이디는 Hugging Face에 로그인한 후 우측의 원 모양을 클릭하면 알 수 있다.

데이터셋을 HF에 Upload

Access Tokens

User Access Tokens

[+ Create](#)

Access tokens authenticate your identity to the Hugging Face Hub and allow applications to perform actions based on token permissions. **Do not share your Access Tokens with anyone;** we regularly check for leaked Access Tokens and remove them immediately.

Name ▾	Value	Last Refreshed Date ▾	Last Used Date ▾	Permissions
🔑 JM_Token1	hf_...bSQt	4 days ago	37 minutes ago	WRITE

- HF Access Token은 HF에 Data를 upload 해야하므로 WRITE 권한이 필요하다

데이터셋을 HF에 Upload



Create a new dataset repository

A repository contains all dataset files, including the revision history.

Owner

JaeminKim



Dataset name

New dataset name

License

License

☒  **Public**
Anyone on the internet can see this dataset. Only you (personal dataset) or members of your

- 우측 메뉴의 New Dataset을 선택하고 왼쪽에 Dataset name에 dataset 이름 부여
- 아래 Public은 모든 사람이 Access 가능
- 이 Dataset 을 먼저 생성한 후에 `upload_file()` 을 나중에 실행해야 한다

Notifications

Inbox (1)

+ New Model

+ New Dataset

+ New Space

+ New Collection

Create organization

Settings

Access Tokens

Billing

Sign Out

데이터셋을 HF에 Upload

The screenshot displays the Hugging Face web interface. At the top, there is a navigation bar with links for Models, Datasets, Spaces, Posts, Docs, Enterprise, and Pricing. A search bar is located on the left. On the right side, a user profile menu is open for 'JaeminKim', showing options like Notifications, Inbox, New Model, New Dataset, New Space, New Collection, Create organization, and Settings. The main content area shows two sections: 'Models' and 'Datasets'. Under 'Models', there is a card for 'JaeminKim/HF3B_callme' updated 4 days ago with 26 downloads. Under 'Datasets', there is a card for 'JaeminKim/callme' updated 4 days ago with 10 views and 3 downloads.

- `upload_file()`을 실행한 후 확인 방법
- 우측 메뉴의 Profile 이름명을 클릭하면 왼쪽 화면과 같이 내가 생성한 Dataset들을 볼 수 있다.

데이터셋을 HF에 Upload

[Dataset card](#) [Viewer](#) [Files and versions](#) [Community 1](#) [Settings](#)

Dataset Viewer

Auto-converted to Parquet [API](#) [Embed](#) [Full Screen View](#)

Split (1)
train · 10 rows

Search this dataset [SQL](#) [Console](#)

instruction string · classes	input string · classes	response string · classes
1 value	10 values	10 values
answer to input	What is the CallMe service?	The CallME service is a delivery service recently launched in Korea.
answer to input	What can I order with CallME?	We can deliver fast food, Japanese food, Chinese food, snacks, food ingredients, general items, etc.
answer to input	How much does the CallME service cost?	The basic fee is 5,000 won, and 1,000 won is added per km

- 특정 dataset을 클릭하면 화면과 같이 그 내용을 볼 수 있다.

Fine Tuning 과정

1. 필요한 Package들 설치하기
2. Dataset 준비하기
3. 모델 훈련(Training) 하기
4. 추론(inference) 하기
5. 모델 저장하기
6. 모델 실행하기

Base 모델 선정, 필요한 값들 설정

```
model_name = 'unsloth/Llama-3.2-3B-Instruct'  
max_seq_length = 2048 # Choose any! We auto support RoPE Scaling internally!  
dtype = None # None for auto detection. Float16 for Tesla T4, V100, Bfloat16 for Ampere+  
load_in_4bit = True # Use 4bit quantization to reduce memory usage. Can be False.
```

- max_seq_length : 모델이 처리할 수 있는 입력 텍스트의 최대 토큰 길이를 설정한다.
- dtype : 모델의 데이터 타입을 지정한다. None으로 설정하면 모델 로드 시 자동으로 하드웨어에 적합한 데이터 타입을 감지한다.
- Float16은 Tesla T4, V100 등에서 사용.
- Bfloat16은 최신 GPU(예: Ampere 아키텍처 이상)에서 사용.
- load_in_4bit : 4-bit 양자화된 모델을 로드할지 여부를 지정한다. True로 설정 시 메모리 사용량이 크게 줄어들며 모델 다운로드 속도가 향상된다. 4-bit 양자화는 성능 저하 없이 모델 크기를 줄이는 데 유용하다.

unsloth

- unsloth는 LoRA 기반의 경량화된 모델 학습 최적화 라이브러리로, 메모리 사용량 절감과 학습 속도 향상을 목표로 설계되었다.
- Gradient Checkpointing 지원 (`use_gradient_checkpointing="unsloth"`)
 - `use_gradient_checkpointing`을 "unsloth"로 설정하면 Unsloth 방식의 메모리 절약 기술을 활용하여 Gradient Checkpointing을 수행한다.
 - 메모리 사용량 절감 (큰 모델을 작은 GPU에서도 훈련 가능)
 - 훈련 속도 향상 (효율적인 연산 적용)
 - Backward 연산 시 필요한 활성화 메모리를 줄임
 - 기본적으로 Hugging Face transformers 라이브러리의 Gradient Checkpointing보다 더 효율적인 방식 사용

unsloth

- Faster LoRA (LoRA 속도 최적화) : LoRA 미세 조정 과정에서 unsloth의 최적화된 행렬 연산 기법을 적용하여 훈련 속도를 증가시킨다.
 - 기존 LoRA보다 최대 2배 빠른 미세 조정
 - 메모리 사용 최적화: GPU VRAM 소비량 절감
 - 미세 조정된 LoRA 가중치를 더 효율적으로 관리
- Paged Attention (페이지드 어텐션) : unsloth는 Paged Attention 기법을 활용하여, 긴 시퀀스 처리 시 메모리 사용량을 줄이고 속도를 향상시킨다.
 - 긴 텍스트도 VRAM 적게 사용하면서 학습 가능
 - Attention 연산을 최적화하여 처리 속도 개선

unsloth

- 더 빠른 4비트 및 8비트 양자화 지원 : LoRA 미세 조정 과정에서 bitsandbytes를 활용한 4-bit 및 8-bit 양자화를 지원하여 메모리 효율성을 높임.
 - QLoRA와 함께 사용 가능
 - 큰 모델도 저사양 GPU에서 훈련 가능
 - 훈련 속도는 유지하면서 메모리 절감

Pre-trained 모델과 tokenizer 설정

```
from unsloth import FastLanguageModel

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = model_name,
    max_seq_length = max_seq_length,
    dtype = dtype,
    load_in_4bit = load_in_4bit
    # token = "hf_...", # use one if using gated model
)
```

```
👉 Unsloth: Will patch your computer to enable 2x faster fr
👉 Unsloth Zoo will now patch everything to make training fa
==((====))== Unsloth 2025.1.5: Fast Llama patching. Transfo
  🐍 🐍 /| GPU: Tesla T4. Max memory: 14.748 GB. Platform
0^0/ 🐍_/ 🐍 Torch: 2.5.1+cu121. CUDA: 7.5. CUDA Toolkit: 1
🐍 / Bfloat16 = FALSE. FA [Xformers = None, FA2 = F
"-_____" Free Apache license: http://github.com/unsloth
Unsloth: Fast downloading is enabled - ignore downloading ba
```

FastLanguageModel.from_pretrained() 메서드 :

- Hugging Face에서 사전 학습된 모델과 토큰라이저를 로드하는 역할을 한다.
- from_pretrained() 메서드를 사용하여 사전 학습된(pretrained) 모델을 로드
- 반환값은 :
 - model : 사전 학습된 언어 모델 객체. 입력 텍스트를 처리하고 결과를 생성한다.
 - tokenizer : 토큰라이저 객체. 입력 텍스트를 토큰화하거나 모델의 출력을 텍스트로 변환하는데 사용된다.

LoRA 기반의 모델을 생성

```
model = FastLanguageModel.get_peft_model(  
    model,  
    r = 16, # Choose any number > 0 ! Suggested 8, 16, 32, 64, 128  
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",  
                     "gate_proj", "up_proj", "down_proj",],  
    lora_alpha = 16,  
    lora_dropout = 0, # Supports any, but = 0 is optimized  
    bias = "none",    # Supports any, but = "none" is optimized  
    # [NEW] "unsloth" uses 30% less VRAM, fits 2x larger batch size  
    use_gradient_checkpointing = "unsloth", # True or "unsloth" for  
    random_state = 3407,  
    use_rslora = False, # We support rank stabilized LoRA  
    loftq_config = None, # And LoftQ
```

- get_peft_model 메서드는 LoRA 기반의 PEFT((Parameters-Efficient Fine-Tuning, 효율적인 미세 조정) 모델을 생성한다.
- 기존 모델에 LoRA 레이어를 추가해, 메모리와 계산 비용을 줄이면서 미세 조정을 수행할 수 있게 만든다

get_peft_model() 매개변수

- $R = 16$
 - LoRA의 저차원 표현 공간의 랭크를 지정한다.
 - 높은 r 값일수록 더 복잡한 표현을 학습할 수 있지만, 메모리와 계산 비용이 증가한다.
 - 일반적으로 추천 값은 8, 16, 32이다.
- `target_modules = ["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj",]`
 - 모델에서 LoRA를 적용할 가중치 모듈(레이어)을 지정합니다.
 - `q_proj`: Query Projection
 - `k_proj`: Key Projection
 - `v_proj`: Value Projection
 - `o_proj`: Output Projection
 - `gate_proj`: Gating Projection
 - `up_proj`, `down_proj`: 모델의 표현력을 높이거나 낮추는 역할

모든 가중치를 업데이트하는 대신, 선택한 `target_modules`에만 LoRA를 적용하여 훈련해야 할 가중치 수를 줄인다. 이를 통해 최적의 성능을 유지하면서도 VRAM 사용량을 크게 줄일 수 있음.

get_peft_model() 매개변수

- `lora_alpha = 16`
 - LoRA 출력값을 스케일링하는 데 사용된다.
 - 높은 `lora_alpha` 값은 출력의 기여도를 높이며, 학습 안정성을 강화할 수 있다.
 - `lora_alpha` 값이 커지면 메모리 사용과 학습 비용이 증가할 가능성이 있다.
- `lora_dropout = 0`
 - LoRA 레이어에서 드롭아웃을 적용할 확률.
 - 0은 드롭아웃 비활성화를 의미합니다.
 - 드롭아웃 비활성화가 일반적으로 더 높은 성능을 제공하지만, 과적합 위험이 증가할 수 있다.
 - 이 경우 학습이 더 안정적이고 빠르게 진행될 수 있지만, 과적합 위험이 증가할 수 있다.

get_peft_model() 매개변수

- `bias = "none"`
 - 모델의 `bias` 추가 여부를 지정한다.
 - `"none"`: `bias`를 추가하지 않는다(권장 옵션).
 - 다른 옵션:
 - `"all"`: 모든 레이어에 `bias`를 추가.
 - `"lora_only"`: LoRA 레이어에만 `bias`를 추가.
- `use_gradient_checkpointing = "unsloth"`
 - GPU 메모리를 절약하기 위해 순전파 결과를 저장하지 않고 역전파 시 재계산한다.
 - `"unsloth"`: `unsloth`의 최적화된 버전을 사용해 VRAM(그래픽 메모리) 사용량을 약 30% 줄이고, 2배 큰 배치 크기를 처리할 수 있도록 지원한다.

get_peft_model() 매개변수

- `random_state = 3407`
 - LoRA 초기화 시 랜덤값 생성에 사용되는 시드 값을 지정한다.
 - 동일한 시드를 사용하면 결과가 재현 가능하다.
- `use_rslora = False`
 - Rank-Stabilized LoRA
 - LoRA의 안정성과 효율성을 높이는 기법.
 - False로 설정하면 일반 LoRA를 사용한다.
 - 사용 목적: Rank가 커져도 weight update scale을 안정적으로 유지.
- `Loftq_config = None`
 - LoftQ(Quantization for LoRA):
 - LoRA 레이어를 추가로 양자화하여 더 작은 메모리 풋프린트를 지원. 학습이 더 오래걸릴 수 있다.
 - unsloth는 이미 QLoRA 최적화가 되어 있기 때문에 사용안함.

Dataset 로드 후 포맷 변환

```
alpaca_prompt = """제공된 dataset은 CalME라서 배달 서비스에 대한 Q&A 입니다.

### Instruction:
{}

### Input:
{}

### Response:
{} """

EOS_TOKEN = tokenizer.eos_token # EOS_TOKEN을 정의해야 합니다. tokenizer는 사

def formatting_prompts_func(examples):
    instructions = examples["instruction"]
    inputs = examples["input"]
    responses = examples["response"]
    texts = []

    for instruction, input, response in zip(instructions, inputs, responses):
        # EOS_TOKEN을 추가하여 생성이 무한히 진행되는 것을 방지합니다.
        text = alpaca_prompt.format(instruction, input, response) + EOS_TOKEN
        texts.append(text)

    return {"text": texts}
```

- Instruction, Input, Response는 각기 모델에게 제공할 지시문, 질문, 응답을 나타내는 자리 표시자(placeholder)이다.{}는 나중에 실제 데이터로 교체될 자리이다.
- EOS_TOKEN은 End of Sequence Token을 나타내는 변수로, 이 토큰은 생성된 텍스트의 끝을 알려준다.
- formatting_prompts_func: 데이터셋의 각 항목을 처리하여 프롬프트 텍스트를 형식에 맞게 변경
- 프롬프트 템플릿에 지시문, 입력, 응답을 각각 채워 넣고 EOS_TOKEN을 추가하여 텍스트 완성

데이터셋 로딩 및 처리

```
from datasets import load_dataset

dataset = load_dataset("JaeminKim/callme", split="train")
dataset = dataset.map(formatting_prompts_func, batched=True, remove_columns=dataset.column_names)

# 첫 번째 예시를 출력하여 확인
print(dataset[0]['text'])
```

call_me.jsonl: 100%  1.76k/1.76k [00:00<00:00, 147kB/s]

Generating train split: 100%  10/10 [00:00<00:00, 213.05 examples/s]

Map: 100%  10/10 [00:00<00:00, 312.20 examples/s]

제공된 dataset은 CallME라서 배달 서비스에 대한 Q&A 입니다. 주어진 데이터를 바탕으로 적절한 응답을 작성하세요

Instruction:
answer to input

Input:
What is the CallMe service?

Response:
The CallME service is a delivery service recently launched in Korea.<|end_of_text|>

- Hugging Face datasets 라이브러리에서 JaeminKim/callme라는 데이터셋을 로드한다.
- split= " train " 은 훈련 데이터를 불러오겠다는 것을 의미한다.
- map 함수는 formatting_prompts_func를 데이터셋의 각 항목에 적용하여 프롬프트 형식으로 변환한다.
- batched=True: 여러 개의 예시를 한 번에 처리하도록 설정한다.
- remove_columns=dataset.column_names: 기존 데이터셋의 열을 삭제하고, instruction, input, response 열만 남긴다.

언어모델 훈련 설정

```
from trl import SFTTrainer
from transformers import TrainingArguments
from unsloth import is_bfloat16_supported

trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = dataset,
    dataset_text_field = "text",
    max_seq_length = max_seq_length,
    dataset_num_proc = 2,
    packing = False, # Can make training 5x faster
    args = TrainingArguments(
        per_device_train_batch_size = 4,
        gradient_accumulation_steps = 4,
        warmup_steps = 5,
        # num_train_epochs = 1, # Set this for
        max_steps = 300,
        learning_rate = 2e-4,
        fp16 = not is_bfloat16_supported(),
        bf16 = is_bfloat16_supported(),
        logging_steps = 1,
        optim = "adamw_8bit",
        weight_decay = 0.01,
        lr_scheduler_type = "linear",
        seed = 3407,
        output_dir = "outputs",
    ),
```

- SFTTrainer: trl 라이브러리의 특수한 트레이너 클래스입니다. 언어 모델을 fine-tune하는 데 사용됩니다.
- max_seq_length: 입력 시퀀스의 최대 길이를 지정한다. 길이가 이보다 길면 자른다.
- dataset_num_proc: 데이터셋을 로드할 때 사용할 프로세스 수.
- packing: True로 설정하면 짧은 시퀀스에서 훈련 속도가 5배 빨라질 수 있다.
- per_device_train_batch_size: 각 장치(GPU/CPU)에서 사용하는 배치 크기
- gradient_accumulation_steps: 경사도를 얼마나 쌓은 후에 가중치를 업데이트할지 설정한다. 이 코드에서는 4로 설정되어 있어, 4번의 스텝 동안 경사도를 누적하고 그 후에 가중치를 업데이트한다
- warmup_steps: 학습 초기에 학습률(learning rate)을 점진적으로 증가시키는 단계
- max_steps: 훈련 스텝의 총 수
- learning_rate: 초기 학습률
- is_bfloat16_supported: 사용 중인 하드웨어가 bfloat16 정밀도를 지원하는지 확인합니다. bfloat16은 일부 하드웨어에서 훈련 속도를 높이는 데 도움이 될 수 있다.

언어모델 훈련 설정

```
from trl import SFTTrainer
from transformers import TrainingArguments
from unsloth import is_bfloat16_supported

trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = dataset,
    dataset_text_field = "text",
    max_seq_length = max_seq_length,
    dataset_num_proc = 2,
    packing = False, # Can make training 5x faster
    args = TrainingArguments(
        per_device_train_batch_size = 4,
        gradient_accumulation_steps = 4,
        warmup_steps = 5,
        # num_train_epochs = 1, # Set this for
        max_steps = 300,
        learning_rate = 2e-4,
        fp16 = not is_bfloat16_supported(),
        bf16 = is_bfloat16_supported(),
        logging_steps = 1,
        optim = "adamw_8bit",
        weight_decay = 0.01,
        lr_scheduler_type = "linear",
        seed = 3407,
        output_dir = "outputs",
    ),
```

- fp16: True로 설정하면 16비트 부동소수점 정밀도(FP16)를 사용한다. 이는 훈련 속도를 높이고 메모리 사용을 줄이는 데 도움이 된다. 이 코드에서는 is_bfloat16_supported()가 False일 경우 FP16을 사용하도록 설정
- bf16: True로 설정하면 bfloat16 정밀도를 사용. bf16이 더 넓은 범위를 가짐. 지수 표현에 8비트 (.vs 5비트)
- optim: 사용하는 옵티마이저. "adamw_8bit"는 8비트 정밀도를 사용하는 AdamW 옵티마이저로, 메모리 사용을 줄이고 대형 모델을 훈련하는 데 유리.
- weight_decay: 옵티마이저에 적용할 가중치 감소(정규화) 비율
- lr_scheduler_type: 학습률 스케줄러의 종류입니다. "linear"는 학습률이 선형적으로 감소하도록 설정.
- seed: 훈련을 재현 가능하게 하기 위한 랜덤 시드.
- output_dir: 훈련 결과와 모델 체크포인트가 저장될 디렉토리

Training 수행

```
import wandb
wandb.init(mode="disabled")
trainer_stats = trainer.train()
```

==((====))== Unsloth - 2x faster free finetuning | Num GPUs = 1
 ### /| Num examples = 10 | Num Epochs = 60
0^0/ ##/ ## Batch size per device = 2 | Gradient Accumulation steps = 4
 ## / Total batch size = 8 | Total steps = 60
"-_____" Number of trainable parameters = 24,313,856
wandb: WARNING The `run\_name` is currently set to the same value as `Train`
[60/60 01:57, Epoch 48/60]

Step	Training Loss
1	3.425500
2	5.841800
3	5.998400
4	5.396900
5	5.000200

- Weights and Biases (W&B) 라이브러리.
- Mode를 disabled로 한 경우는 훈련 로그와 메트릭 데이터를 자동으로 서버에 업로드 하지 않는 경우
- `trainer.train()`은 훈련을 수행하고, 훈련 결과(예: 학습한 에폭 수, 훈련 손실 값, 정확도 등)를 반환한다.

Inference 수행

```
# alpaca_prompt 정의
alpaca_prompt = """제공된 dataset은 CallME라서 배달 서비스에 대한 Q

### Instruction:
{instruction}

### Input:
{input}

### Response:
"""

# FastLanguageModel 설정 (이 부분은 그대로 유지)
FastLanguageModel.for_inference(model)

# 원본 dataset 로드
original_dataset = load_dataset("JaeminKim/callme", split="train")
```

- alpaca_prompt는 Alpaca 모델에서 사용하는 지침 기반 프롬프트 형식을 따라서 생성.
- FastLanguageModel은 사전 학습된 모델을 효율적으로 추론할 수 있도록 설정한다.
- for_inference(model)은 모델이 추론(inference) 모드로 작동하도록 준비시킨다.
- 마지막은 dataset 다시 로드

Inference 수행

```
# dataset에서 무작위로 하나의 예제 선택
import random
random_index = random.randint(0, len(original_dataset) - 1)
example = original_dataset[random_index]

# 입력 준비
instruction = example['instruction']
input_text = example['input']

# 토큰화
inputs = tokenizer(
    [
        alpaca_prompt.format(
            instruction=instruction,
            input=input_text
        )
    ],
    return_tensors="pt"
).to("cuda")
```

- Random으로 하나의 예제를 선택해서 instruction, input_text에 배정.
- tokenizer() : 입력된 텍스트를 모델이 이해할 수 있는 숫자 토큰으로 변환합니다.
- return_tensors="pt" : 토큰라이저가 PyTorch 텐서 형태로 반환하도록 지정한다. 텐서란 GPU/CPU에서 수학적 연산을 효율적으로 처리할 수 있는 데이터 구조.
- CUDA(Compute Unified Device Architecture)는 NVIDIA가 개발한 GPU 컴퓨팅 플랫폼이다.
- GPU는 대량의 병렬 연산을 수행하는 데 최적화되어 있어, 딥러닝 모델의 훈련 및 추론 속도를 크게 향상시킨다.
- .to("cuda")는 데이터를 CPU에서 GPU로 옮긴다. 만약 GPU가 없으면 에러가 발생

Inference 수행

```
# 생성
outputs = model.generate(**inputs, max_new_tokens=64, use_cache=True)

# 디코딩 및 출력
generated_text = tokenizer.batch_decode(outputs, skip_special_tokens=True)[0]
```

- `model.generate()`는 자연어 생성 모델에서 텍스트를 생성하는 메서드이다. 입력 텍스트(`inputs`)에 기반하여 이어질 텍스트를 예측한다. 모델은 입력 시퀀스의 확률 분포를 기반으로 다음 토큰(단어 또는 하위 단위)을 점진적으로 생성한다.
- `**inputs` : Dictionary 키-값 쌍을 풀어서 `generate` 메서드에 전달한다.
- `inputs`는 GPU에 저장된 PyTorch 텐서로, 모델이 이해할 수 있는 형식의 입력 데이터이다.
- `max_new_tokens=64` : 생성되는 텍스트의 최대 토큰 수를 64로 제한한다. 영어로는 대략 48단어.
- `use_cache=True` : 모델이 이전 계산 결과(캐시)를 저장하고 재활용하여 속도를 높인다.
- `outputs`는 모델이 생성한 텍스트의 토큰 ID로 이루어진 텐서를 포함한다.
- `tokenizer.batch_decode()`는 모델이 생성한 토큰 ID를 사람이 읽을 수 있는 텍스트로 변환한다. 출력은 디코딩된 문자열이다.
- `skip_special_tokens=True` : 모델이 출력하는 특별 토큰(예: `<pad>`, `<s>`, `</s>`)을 제거한다.

Inference 수행

```
# TextStreamer 설정  
text_streamer = TextStreamer(tokenizer)
```

- TextStreamer()는 generate 메서드로 텍스트를 생성하는 동안, 각 토큰이 생성될 때마다 즉시 출력된다. 사용자는 생성 과정의 중간 결과를 실시간으로 확인할 수 있다.

Inference 수행

생성된 응답:

제공된 dataset은 CalIME라서 배달 서비스에 대한 Q&A 입니다. 주어진 데이터를 바탕으로 적절한 응답을 작성하세요.

Instruction:

answer to input

Input:

What is the price of CalIME service?

Response:

The basic fee is 5,000 won, and 1,000 won is added per km

생성된 응답:

제공된 dataset은 CalIME라서 배달 서비스에 대한 Q&A 입니다. 주어진 데이터를 바탕으로 적절한 응답을 작성하세요.

Instruction:

answer to input

Input:

Please explain about Seoul

Response:

Seoul is the capital city of Korea, located in the southwestern part of the Korean Peninsula.

- Fine Tuning 에서 나온 결과값과 일반 Q&A 결과값 확인.

모델 저장

```
save_path = "fine-tuned-callcrazy-model"

# LoRA 적용된 모델 저장
trainer.model.save_pretrained(save_path)

# tokenizer 저장
trainer.processing_class.save_pretrained(save_path)

# Hugging Face Hub 업로드
repo_name = "JaeminKim/HF3B_callme"
api = HfApi()
api.create_repo(repo_name, exist_ok=True)

trainer.model.push_to_hub(repo_name) # Model 업로드
trainer.processing_class.push_to_hub(repo_name) #
```

- "fine-tuned-callcrazy-model" : 로컬 서버에 저장할 디렉토리명.
- `trainer.model.save_pretrained(save_path)` : 학습된 모델을 지정한 경로에 저장
- `trainer.processing_class.save_pretrained(save_path)` : 토큰나이저도 함께 저장
- `trainer.model.push_to_hub(repo_name)` : Fine-Tuning된 모델을 Hugging Face에 업로드
- `trainer.processing_class.push_to_hub(repo_name)` : 토큰나이저도 함께 업로드

모델 저장

Users...

Models Datasets Spaces Posts Docs Enterprise Pricing



Create a new model repository

A repository contains all model files, including the revision history.

Owner

JaeminKim

Model name

New model name

Profile

JaeminKim

Notifications

Inbox (1)

+ New Model

+ New Dataset

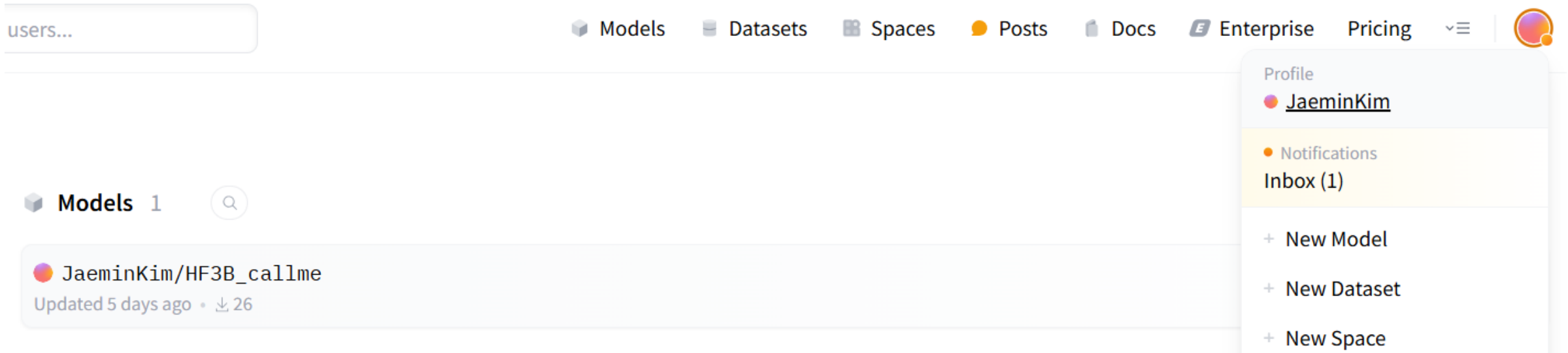
+ New Space

+ New Collection

Create organization

- `model.push_to_hub_gguf()`는 실행하기 전에 먼저 오른쪽의 New Model 메뉴를 선택해서 새로운 모델 저장소를 만든다.

모델 저장



- 오른쪽에 Profile 명을 클릭하면 Hugging Face에 저장된 모델을 확인할 수 있다.

모델 저장

 **Hugging Face**

 **JaeminKim** / **HF3B_callme**   like 0

 Safetensors  unsloth  License: apache-2.0

 Model card  **Files and versions**  Community  Settings

 main  **HF3B_callme**

 **JaeminKim** Upload model trained with Unsloth **145e4db** **VERIFIED**

 .gitattributes 	1.57 kB	
 README.md 	49 Bytes	
 adapter_config.json 	817 Bytes	
 adapter_model.safetensors  	97.3 MB 	

- 필요하면 오른쪽에 다운로드 버튼을 눌러서 파일을 PC에 다운로드 할 수 있다.

Fine Tuning 모델 사용하기

모델을 로드

```
# Unsloth가 Hub repo에서 베이스 모델 + LoRA 어댑터를 자동으로 결합
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="JaeminKim/HF3B_callable", # Fine-tuned repo
    max_seq_length=4096,
    dtype=None, # None → bf16/fp16 자동 감지
    load_in_4bit=True, # 학습 시와 동일 설정 유지
)

# 추론 모드 활성화 (Unsloth 내부 최적화 커널 적용)
FastLanguageModel.for_inference(model)

# device 확인만 (이동은 하지 않음)
device = "cuda" if torch.cuda.is_available() else "cpu"
```

- Hugging Face에 저장했던 모델과 tokenizer를 그대로 로드
- 추론 모드 활성화
- GPU가 있으면 cuda로 설정

모델 테스트

```
def generate_response(prompt, max_length=512):
    inputs = tokenizer(prompt, return_tensors="pt").to(device)

    with torch.no_grad():
        output = model.generate(
            **inputs,
            max_length=max_length,
            eos_token_id=tokenizer.eos_token_id,
            temperature=0.7, # 더 다양한 답변 생성
            top_p=0.9, # 상위 90% 확률의 단어만 선택하여 일반적
        )

    return tokenizer.decode(output[0], skip_special_tokens=True)
```

- `tokenizer(prompt, return_tensors="pt").to(device)` : 입력된 prompt(텍스트)를 토큰(token)으로 변환
- `return_tensors="pt"` 옵션은 PyTorch(`torch.Tensor`) 형식으로 변환
- `.to(device)` device(GPU 또는 CPU)로 데이터를 이동하여 더 빠른 계산이 가능하도록 설정
- `torch.no_grad()` : 그래디언트(Gradient) 계산 비활성화
- `model.generate()` : 모델이 새로운 답변을 생성
- `tokenizer.decode()` : 토큰을 사람이 읽을 수 있는 문장으로 변환

모델 테스트

```
instruction = "Answer to Input"
input_text = "What is CallCrazy service?"

alpaca_prompt = """제공된 dataset은 CallCrazy 서비스입니다.

### Instruction:
"Answer to Input"

### Input:
"What is CallCrazy service?"

### Response:
"""

response = generate_response(alpaca_prompt)
print(response)
```

```
### Instruction:
"Answer to Input"

### Input:
"What is CallCrazy service?"

### Response:
The CallCrazy service is a delivery service recently launched in Korea.
```

```
instruction = "Answer to Input"
input_text = "What is the price of CallCrazy service?"

alpaca_prompt = """제공된 dataset은 CallCrazy라서 배달료를 더합니다.

### Instruction:
"Answer to Input"

### Input:
"What is the price of CallCrazy service?"

### Response:
"""

response = generate_response(alpaca_prompt)
print(response)
```

```
### Instruction:
"Answer to Input"

### Input:
"What is the price of CallCrazy service?"

### Response:
The basic fee is 5,000 won, and 1,000 won is added per km.
```

A young man with dark hair and glasses is shown in profile, looking intently at a computer screen. The background is a blurred classroom with other students at their desks. The text 'intel digital readiness' is overlaid on the left side of the image. A white curved line separates the text area from the man's face. A solid green rectangle is in the bottom-left corner.

intel. digital readiness